

Mobile Development :

4 : Interactivity with Flutter for Mobile Development

Interactivity on Flutter screens



Professor Imed Bouchrika

National School of Artificial Intelligence
imed.bouchrika@ensia.edu.dz

Outline :

- **Section 1 : Revision**
 - Flutter Widgets
 - Stateless & Stateful
- **Section 2 : Stateful Widgets & Interactivity**
 - Counter App
 - Move Single Square
 - Water Sorting Puzzle
 - Tic Tac Toe Game
 - Todo App
- **Section 3 : Navigation**
 - Navigating between screens
 - Passing data

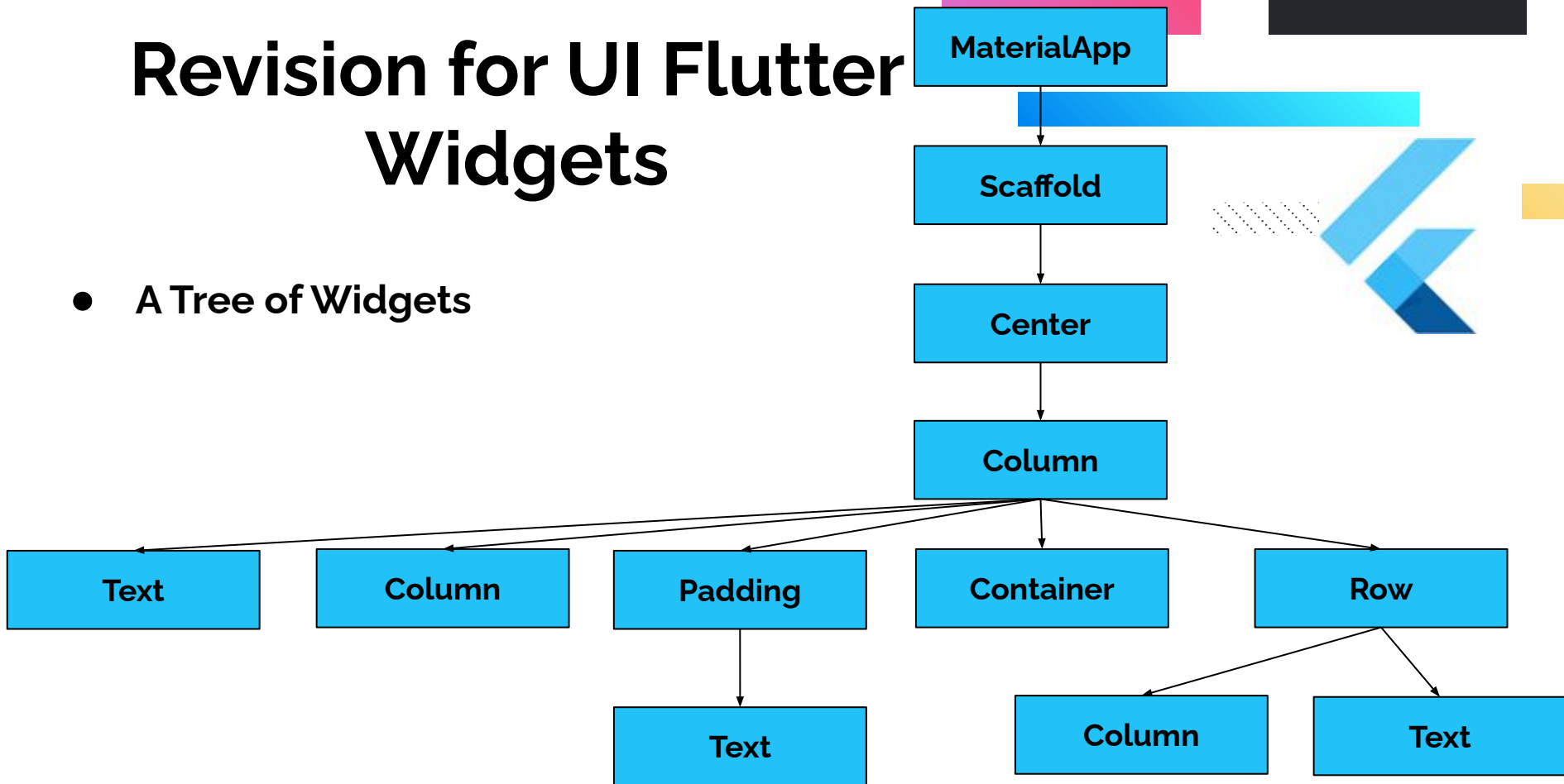
Section 1

**Revision : Constructing UI using
Flutter Widgets**



Revision for UI Flutter Widgets

- A Tree of Widgets

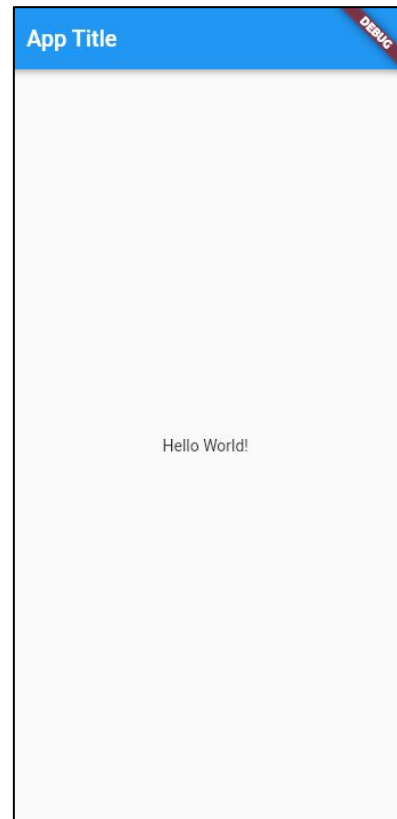


Revision for UI Flutter Widgets

- **Scaffold** : appBar, body, floatingActionButton, drawer, bottomNavigationBar
- **AppBar** : title, leading, elevation
- **Container** : child, padding, margin, height, width, decoration alignment
- **Column** : children, mainAxisAlignment, crossAxisAlignment
- **Row** : children, mainAxisAlignment, crossAxisAlignment
- **Stack** : alignment, children,
- **SizedBox** : child, height, width,
- **Card** : child, color, elevation, margin, clipBehavior, shape
- **Expanded** : child, flex
- **Spacer** : flex
- **Padding** : padding --> EdgeInsets.all|only|fromLTRB|symmetric
- **Center** : child
- **Align** : child, alignment
- **ListView.builder** : scrollDirection, physics, itemCount, itemBuilder(context, index)
- **GridView.builder** : padding, physics, itemCount, itemBuilder(context, index)
- **Text** : style, textAlign
- **TextFormField** : onChanged, controller, decoration, validator
- **ElevatedButton** : style, onPressed, child
- **IconButton** : icon, onPressed
- **Icon** : color, size,
- **Slider** : value, max, divisions, onChanged, label
- **Switch** : value, onChanged, thumbIcon
- **ListTile** : leading, title, subtitle, trailing, tileColor, onTap
- **Image.asset** : imagePath,
- **TextStyle** : fontSize, color, fontWeighth
- **BoxDecoration** : image, shape, border, borderRadius, color
- **Opacity** : opacity, child
- **RoundedRectangleBorder** : borderRadius, side
- **Position** : left, bottom, right, top
- **BoxShadow** : color, offset, blurStyle

Revision for UI Flutter Widgets

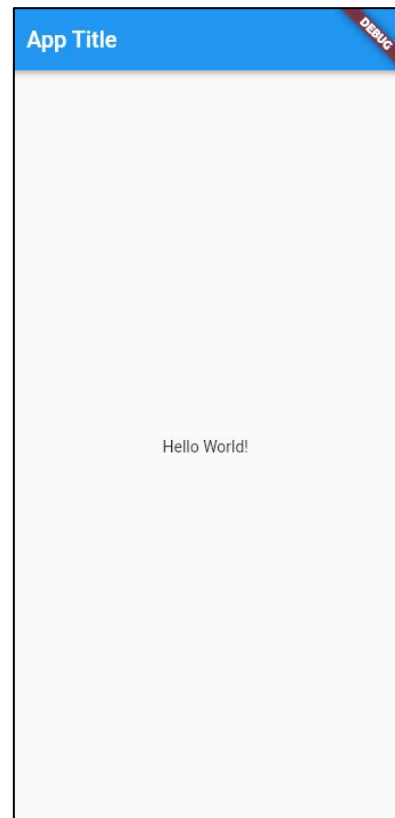
- Tree of Widgets ?



Revision for UI Flutter Widgets

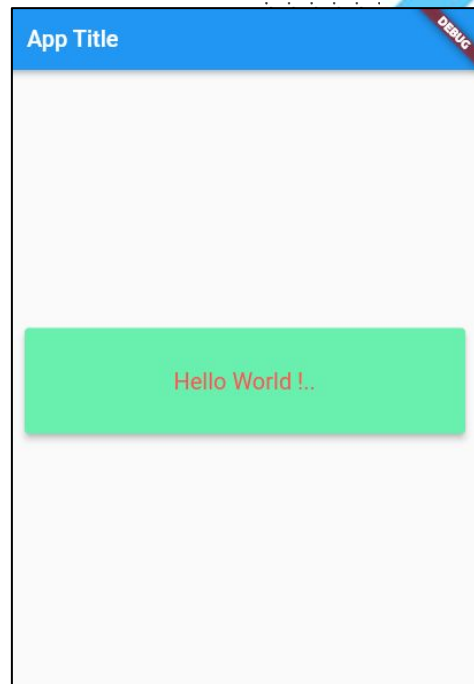
- Skeleton for Building an App

```
import 'package:flutter/material.dart';  
void main() {  
  runApp(MaterialApp(  
    home: Scaffold(  
      appBar: AppBar(title: const Text('App Title')),  
      body: const Center(  
        child: Text('Hello World!'),  
      ),  
    ),  
  ));  
}
```



Revision for UI Flutter Widgets

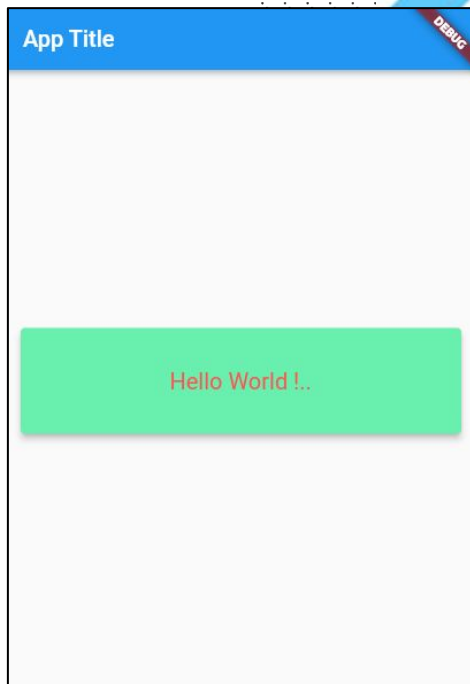
- Skeleton for Building an App : Styling



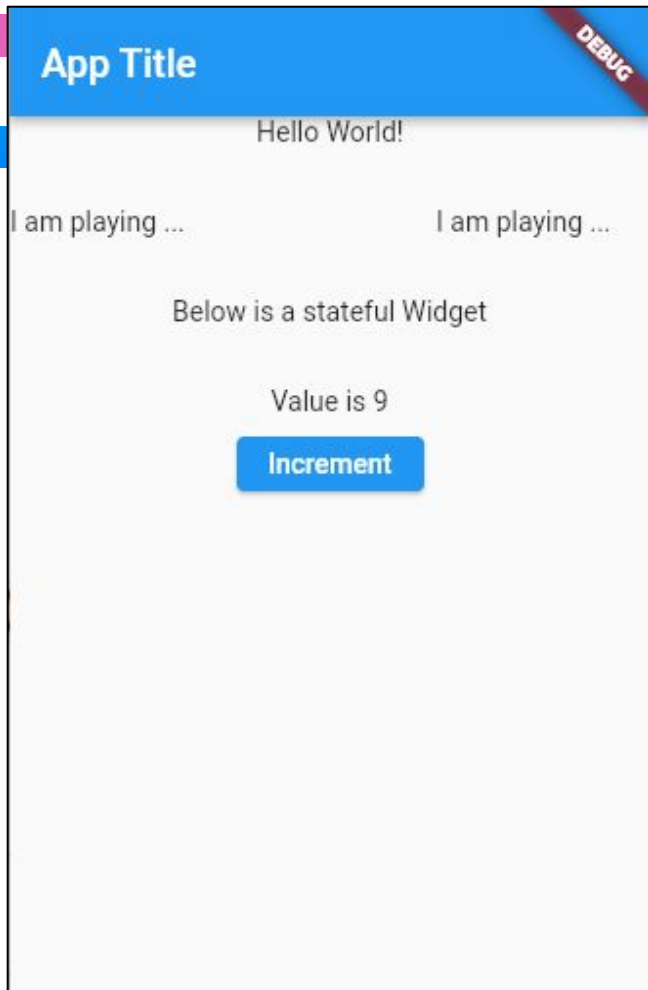
Revision for UI Flutter

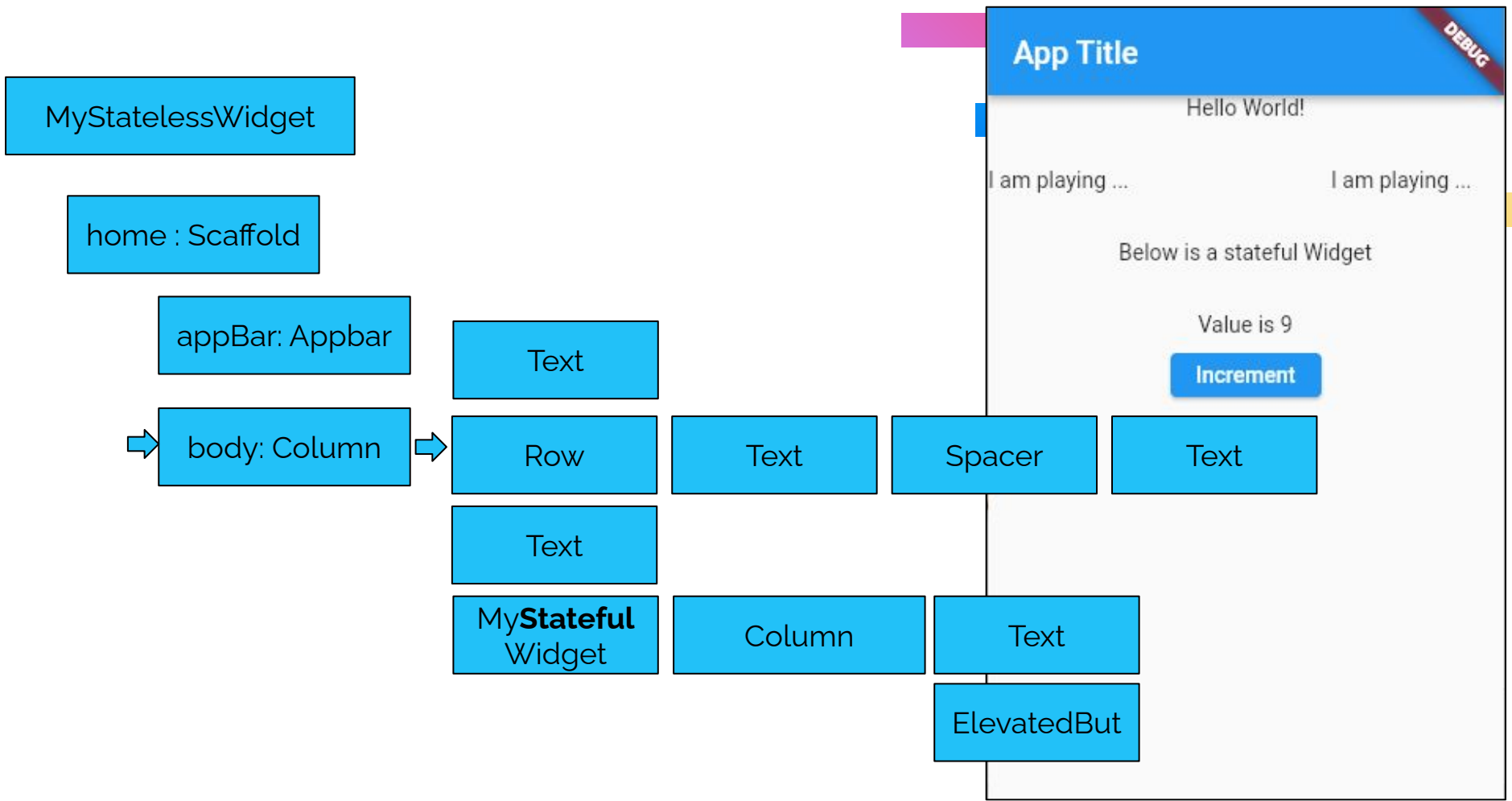
```
import 'package:flutter/material.dart';

void main() {
  runApp(MaterialApp(
    home: Scaffold(
      appBar: AppBar(title: const Text('App Title')),
      body: Center(
        child: Container(
          margin: EdgeInsets.all(10),
          width: double.infinity,
          height: 100,
          child: Card(
            color: Colors.greenAccent,
            elevation: 5,
            child: Center(
              child: Text(
                'Hello World !..',
                style: TextStyle(
                  fontSize: 20,
                  color: Colors.redAccent),
              ),
            ),
          ),
        ),
      ),
    ),
  ));
}
```

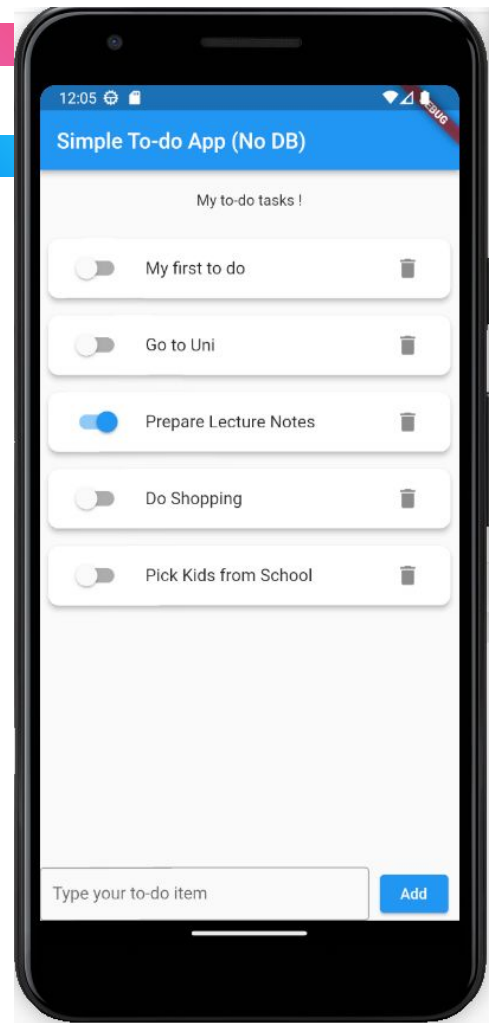


Revision for UI Flutter Widgets



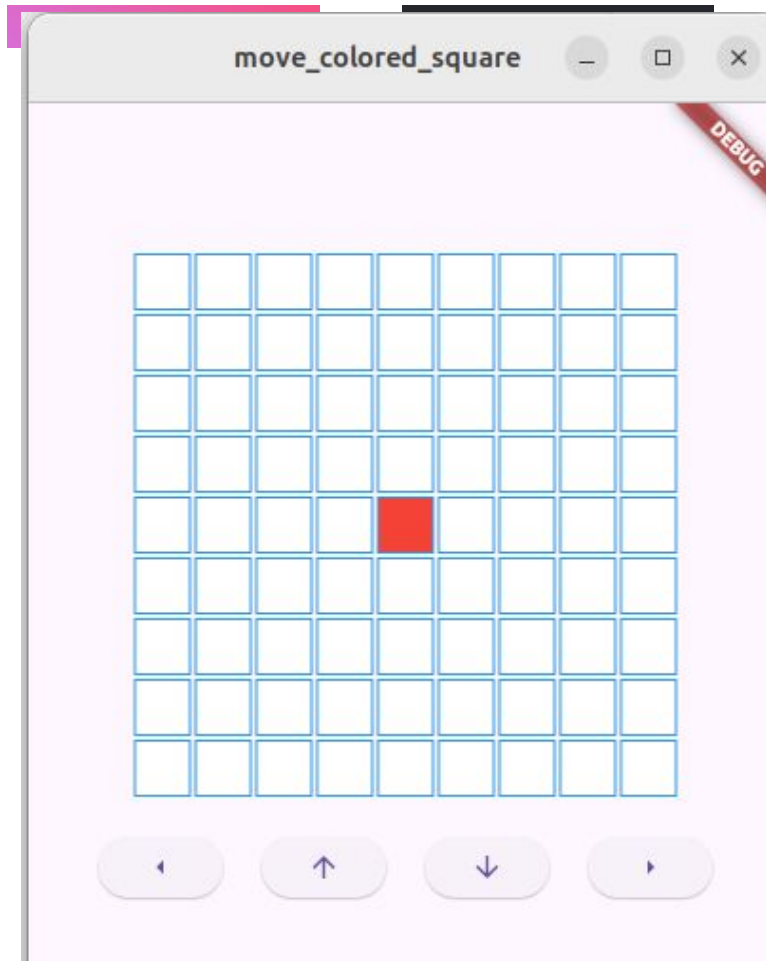


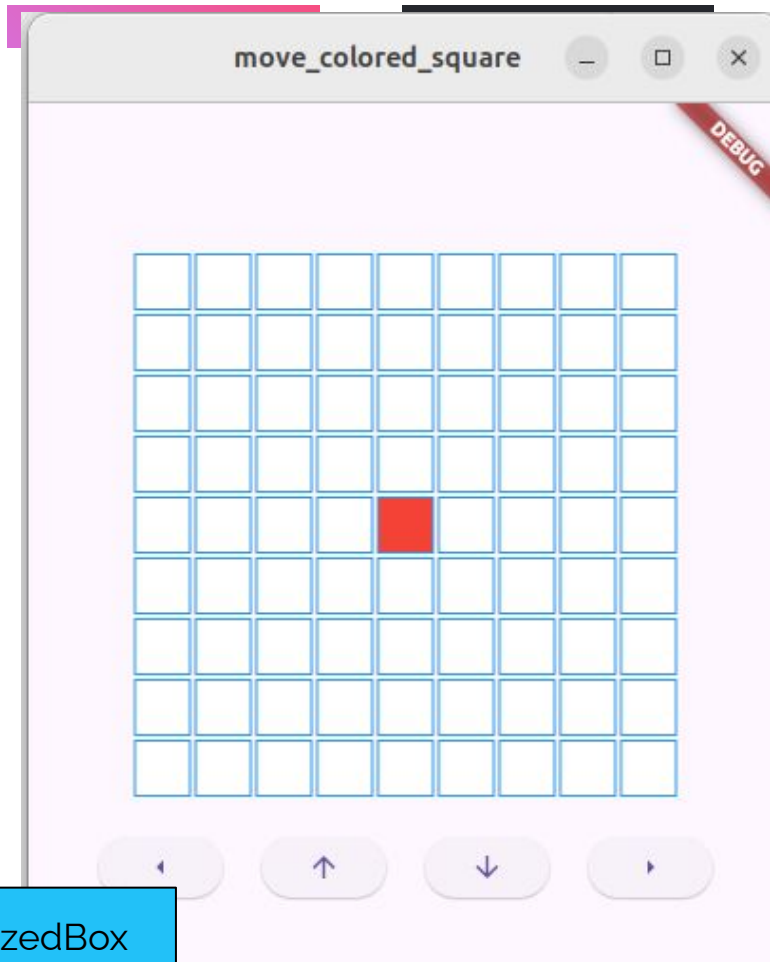
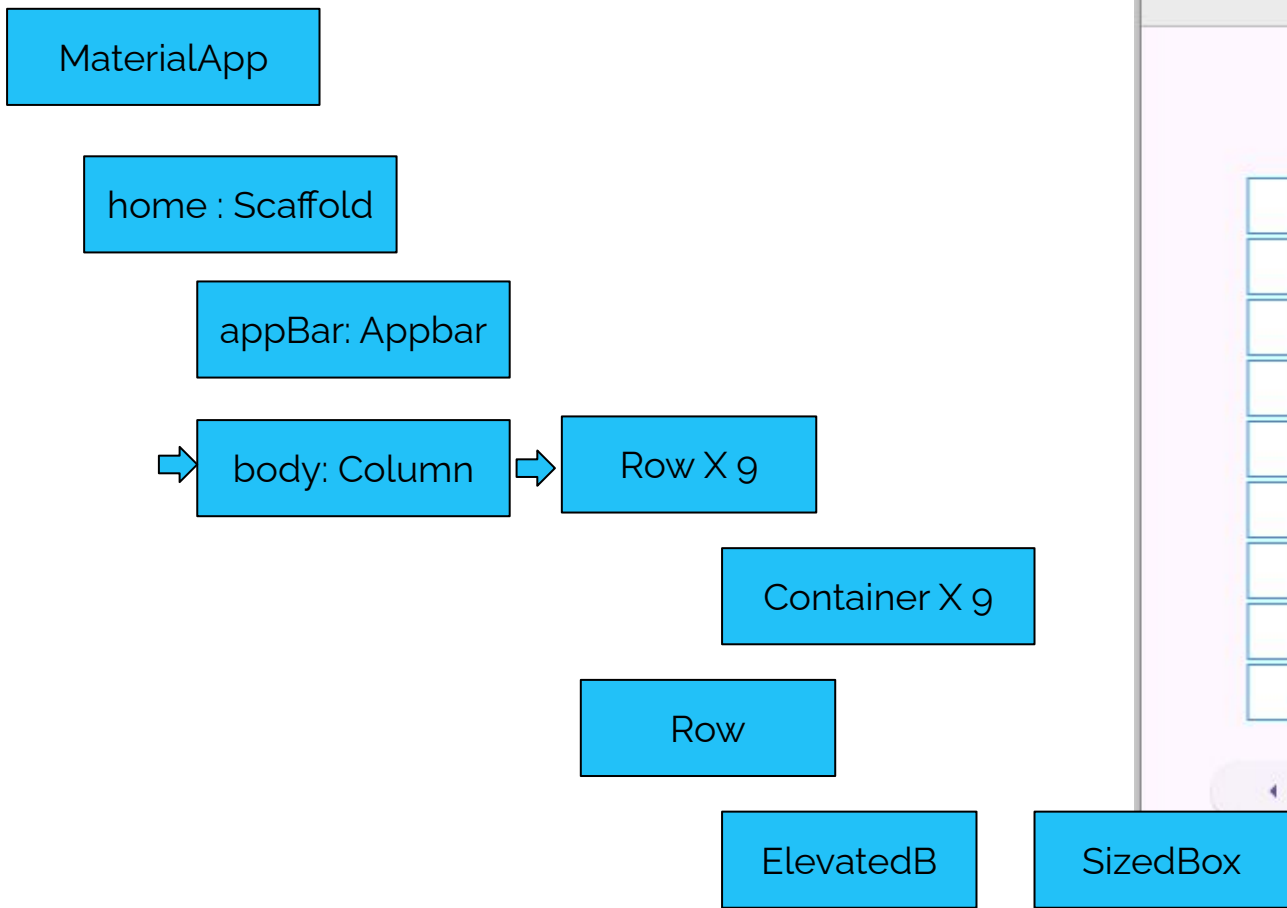
Revision for UI Flutter Widgets



Revision for UI Flutter Widgets

- Tree of Widgets :

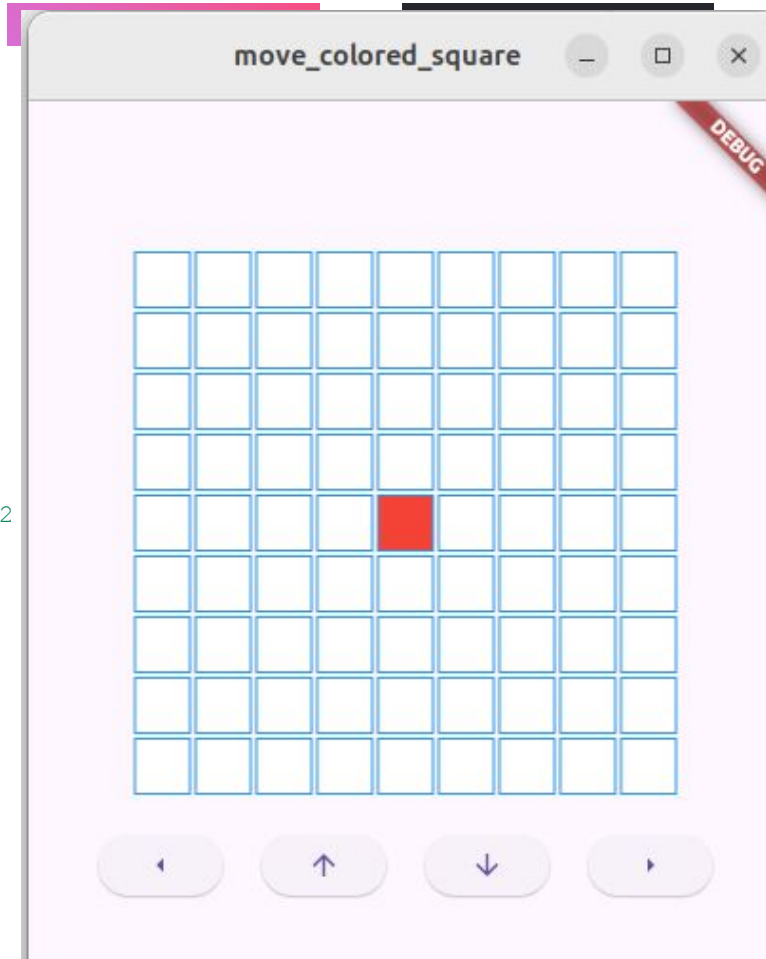




```

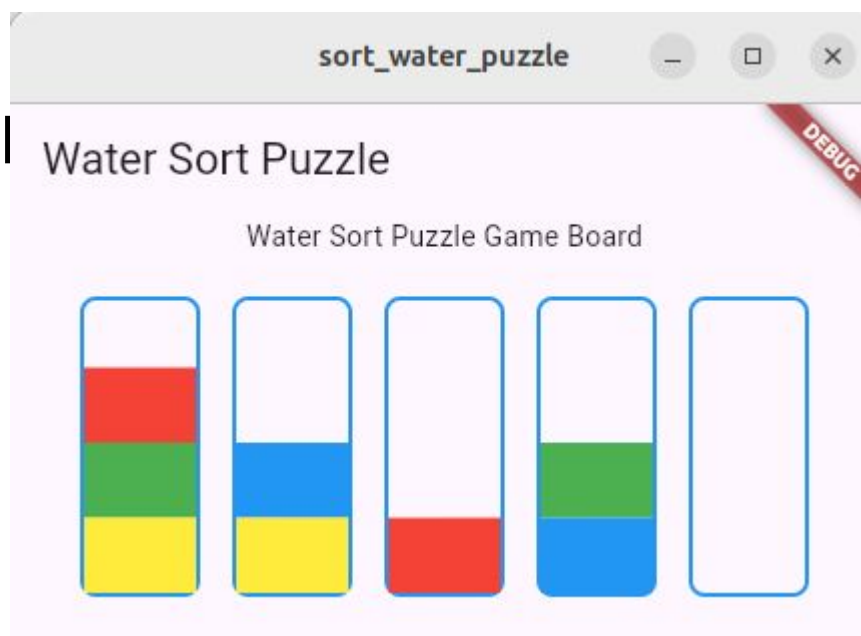
return Scaffold (
  body: Center (
    child: Column (
      mainAxisAlignment: MainAxisAlignment.center,
      crossAxisAlignment: CrossAxisAlignment.center,
      children: [
        for (int r = 0; r < board_size; r++)
          Row (
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              for (int c = 0; c < board_size; c++)
                Container (
                  width: board_cell_width,
                  height: board_cell_height,
                  margin: const EdgeInsets.all(1),
                  decoration: BoxDecoration (
                    color:
                      r == board_size ~/ 2 && c == board_size ~/ 2
                        ? Colors.red
                        : Colors.white,
                    border: Border.all(color: Colors.blue),
                  ),
                ),
              ),
            ],
          ),
        SizedBox (height: 20),
        Row (
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            ElevatedButton (
              onPressed: () {},
              child: const Icon(Icons.arrow_left),
            ),
            SizedBox (width: 20),
            ElevatedButton (

```



Revision for UI Flutter Widgets

- Tree of Widgets :



MaterialApp

home : Scaffold

appBar: AppBar

Text



body: Column

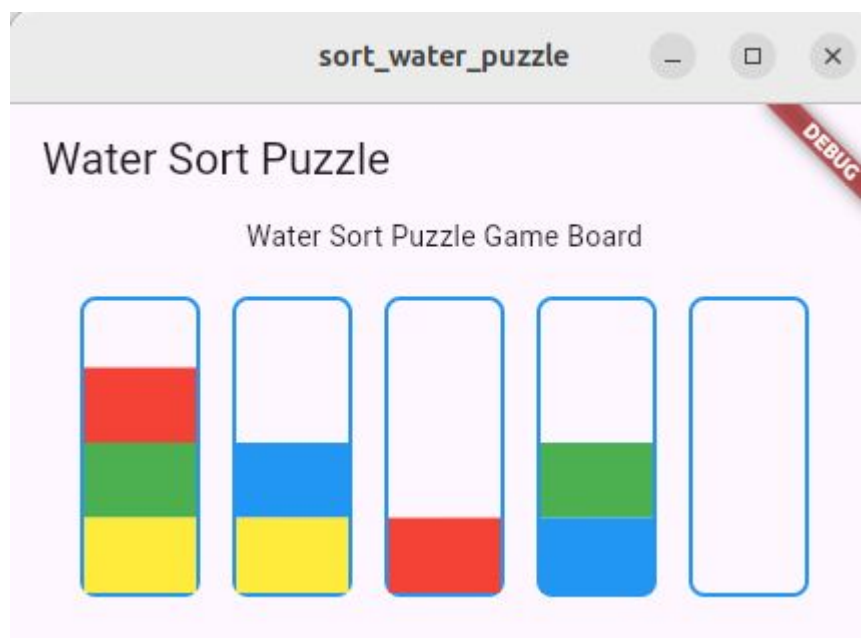


Row X **N**

Container

Column

Container

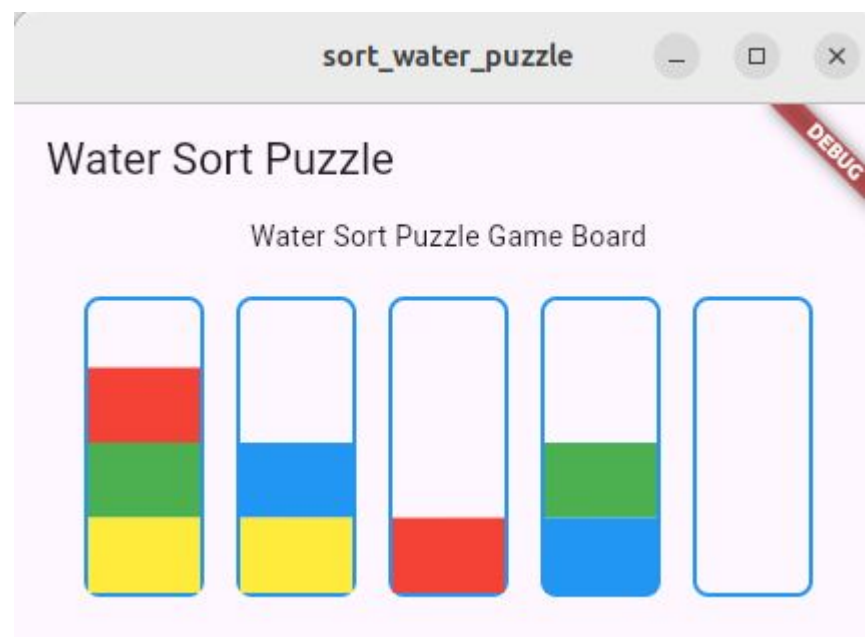


```

List bars = [
  [Colors.red, Colors.green],
  [Colors.blue, Colors.yellow],
  [Colors.red],
  [],
];

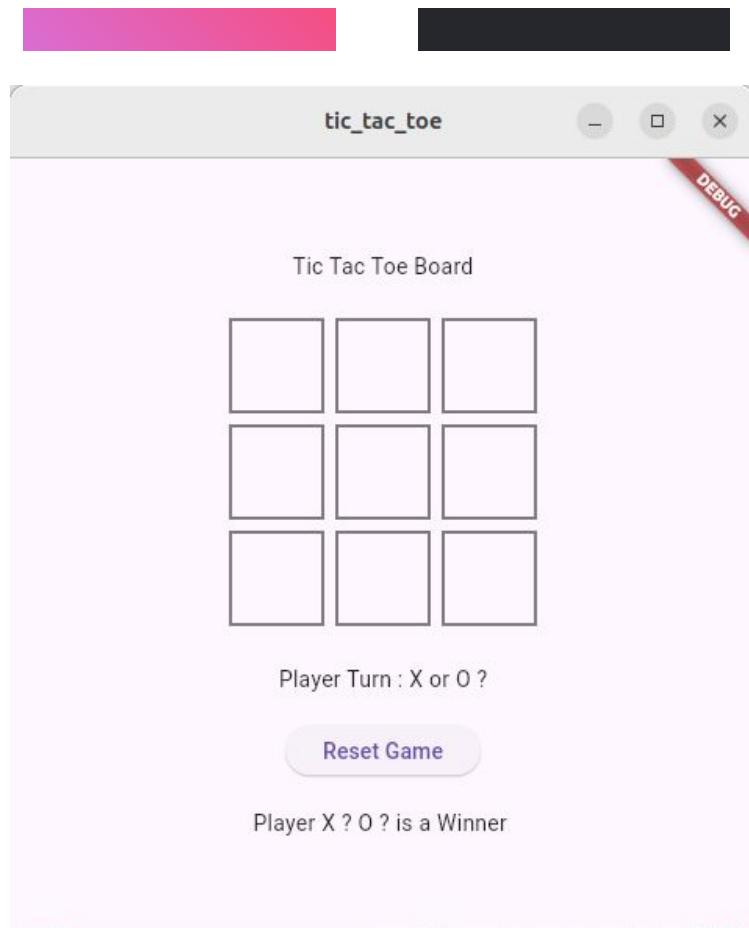
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Water Sort Puzzle')),
    body: Center(
      child: Column(
        children: [
          Text('Water Sort Puzzle Game Board'),
          SizedBox(height: 20),
          Row(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              for (int b = 0; b < bars.length; b++)
                Container(
                  width: 60,
                  margin: EdgeInsets.symmetric(horizontal: 8),
                  height: 150,
                  decoration: BoxDecoration(
                    border: Border.all(color: Colors.blue, width: 2),
                    borderRadius: BorderRadius.circular(8),
                  ),
                  child: Column(
                    mainAxisAlignment: MainAxisAlignment.end,
                    children: [
                      for (int i = 0; i < bars[b].length; i++)
                        Container(width: 60, height: 150 / 4, color: bars[b][i]),
                    ],
                  ),
                ),
            ],
          ),
        ],
      ),
    ),
  );
}

```



Revision for UI Flutter Widgets

- Tree of Widgets :



MaterialApp

home : Scaffold

appBar: AppBar

body: Column

Text

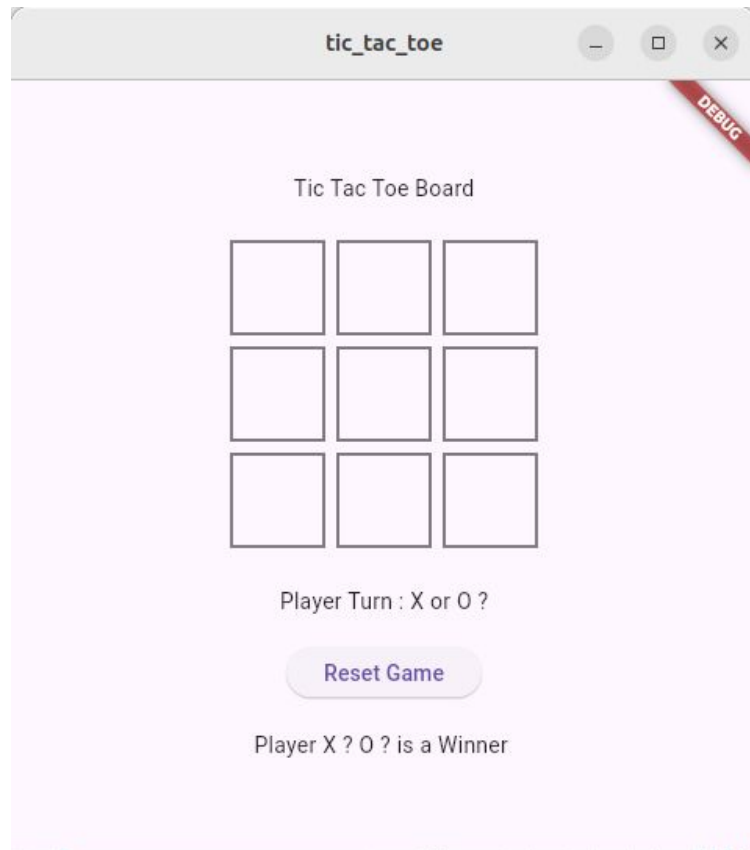
Row X 3

Cell X 3

Text

Button

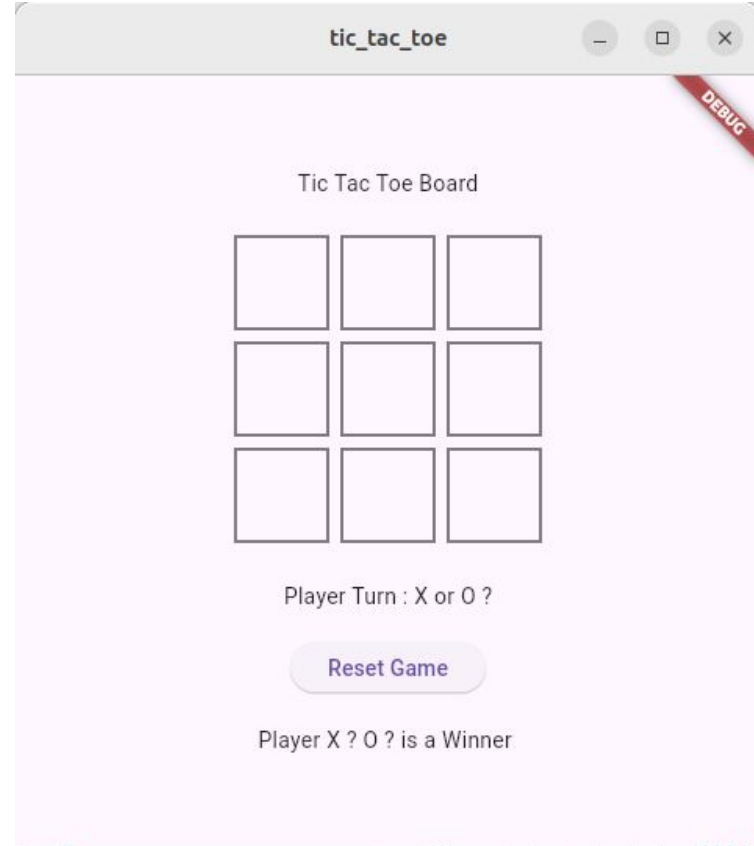
Text



```

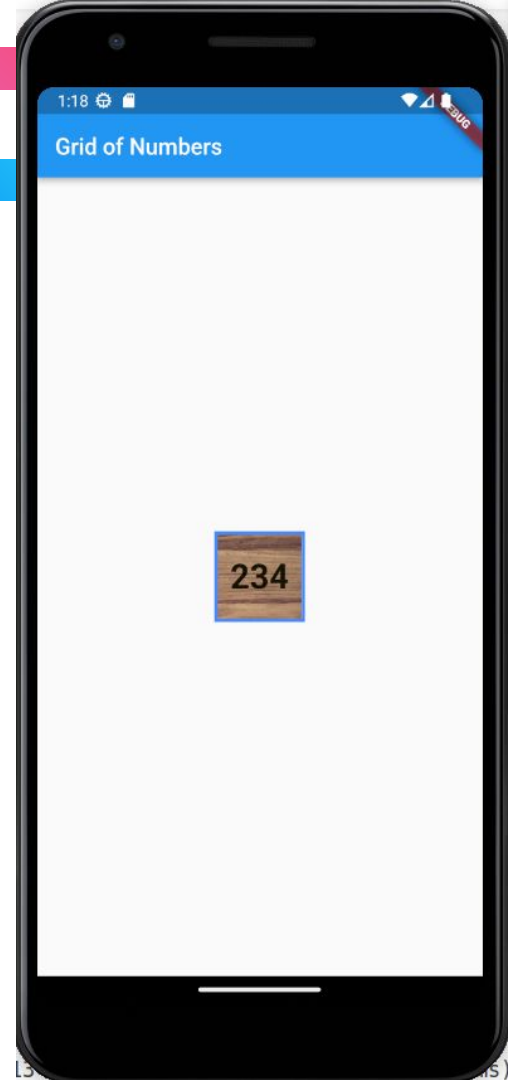
@override
Widget build(BuildContext context) {
  return Scaffold(
    body: Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          Text('Tic Tac Toe Board'),
          SizedBox(height: 20),
          for (int i = 0; i < 3; i++)
            Row(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                for (int j = 0; j < 3; j++)
                  Container(
                    width: 60,
                    height: 60,
                    margin: EdgeInsets.all(4),
                    decoration: BoxDecoration(
                      border: Border.all(color: Colors.black),
                    ),
                    child: Center(child: Text('')),
                  ),
                ],
            ),
          SizedBox(height: 20),
          Text('Player Turn : X or O ?'),
          SizedBox(height: 20),
          ElevatedButton(onPressed: () {}, child: Text('Reset Game')),
          SizedBox(height: 20),
          Text('Player X ? O ? is a Winner '),
        ],
      ),
    ),
  );
}

```



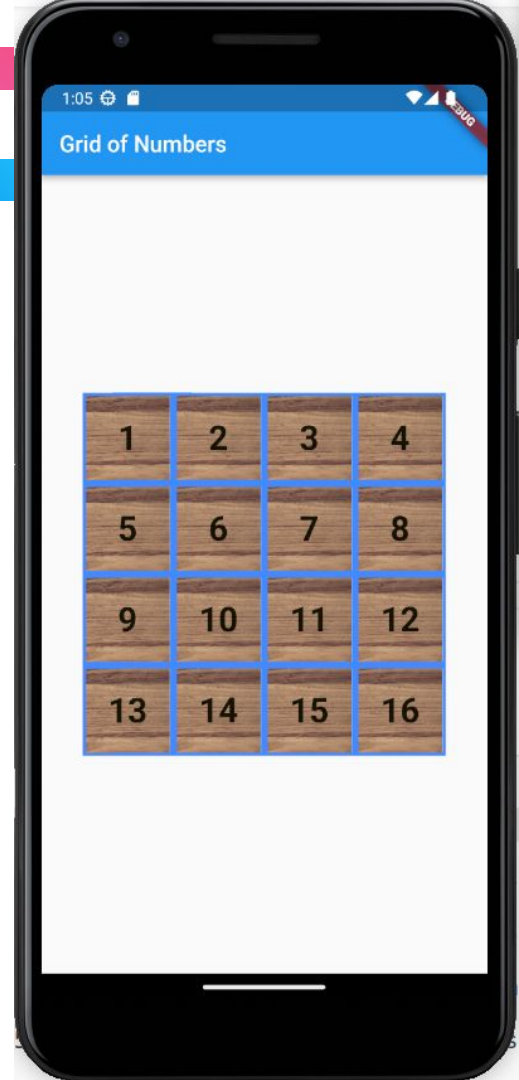
Revision for UI Flutter Widgets

- Tree of Widgets :



Revision for UI Flutter Widgets

- Tree of Widgets :



MaterialApp

home : Scaffold

appBar: AppBar



body: Column



Row X 4

Cell X 4

1:05

Grid of Numbers

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Section 2

Stateful Widgets & Interactivity



Stateless & Stateful Widgets for Flutter



- **Stateful Widget :**
 - They can change based on the interaction with the user. (i.e they are mutable)



Simple Counter

Stateless & Stateful Widgets for Flutter

- **Stateful Widget :**

- They can change based on the interaction with the user. (i.e they are mutable)



```

import 'package:flutter/material.dart';

class MyStatefulExample extends StatefulWidget {
  const MyStatefulExample({super.key});

  @override
  _MyStatefulExampleState createState() => _MyStatefulExampleState();
}

class _MyStatefulExampleState extends State<MyStatefulExample> {
  int _counter = 0;

  @override
  Widget build(BuildContext context) {
    return Column(
      children: <Widget>[
        SizedBox(height: 100),
        Text('Counter: $_counter'),
        ElevatedButton(
          onPressed: () {
            setState(() {
              _counter = _counter + 1;
            });
          },
          child: Text('Increment'),
        ),
      ],
    );
  }
}

```

```

Run | Debug | Profile
4 void main() {
5   | runApp(MainApp());
6 }
7
8 class MainApp extends StatelessWidget {
9   const MainApp({super.key});
10
11   @override
12   Widget build(BuildContext context) {
13     return MaterialApp(
14       home: Scaffold(
15         body: Center(
16           child: MyStatefulExample(),
17         ), // Center
18       ), // Scaffold
19     ); // MaterialApp
20   }
21 }

```



Stateless & Stateful Widgets for Flutter

You create variables, data collections, objects and you attach them to widget/State,

Later, You change the data or variable values and you **setState** to refresh the screen

Stateless & Stateful Widgets for Flutter

- Stateful Widget

The stateful widget Constructor takes another **real** argument to be used.

```
import 'package:flutter/material.dart';

class MyStatefulWidget extends StatefulWidget {
  var myStart = 0;

  MyStatefulWidget({super.key, this.myStart = 0});

  @override
  _MyStatefulWidgetState createState() => _MyStatefulWidgetState();
}

class _MyStatefulWidgetState extends State<MyStatefulWidget> {
  @override
  Widget build(BuildContext context) {
    return Text('Value is ${widget.myStart}');
  }
}
```

Stateless & Stateful Widgets for Flutter

- Stateful Widget

To access data of the Widget from the state instance, use :
widget.varName

```
import 'package:flutter/material.dart';

class MyStatefulWidget extends StatefulWidget {
  var myStart = 0;

  MyStatefulWidget({super.key, this.myStart = 0});

  @override
  _MyStatefulWidgetState createState() => _MyStatefulWidgetState();
}

class _MyStatefulWidgetState extends State<MyStatefulWidget> {
  @override
  Widget build(BuildContext context) {
    return Text('Value is ${widget.myStart}');
  }
}
```


Stateless & Stateful Widgets for

- Stateful Widget

Stateless and Stateful Widgets are integrated into the UI by calling the constructor.

No argument is given for the stateful Widget

```
import 'package:flutter/material.dart';
import
'package:reminder_flutter_hello_world_two/widgets/myStatefulWidget.d
art';
import
'package:reminder_flutter_hello_world_two/widgets/myWidget.dart';

void main() {
  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: const Text('App Title')),
        body: Center(
          child: Column(children: [
            const Text('Hello World!'),
            const SizedBox(height: 30),
            const myWidget(),
            const SizedBox(height: 30),
            const Text('Below is a stateful Widget'),
            const SizedBox(height: 30),
            MyStatefulWidget(),
          ])),
        ),
      );
  }
}
```

Stateless & Stateful Widgets for

- Stateful Widget

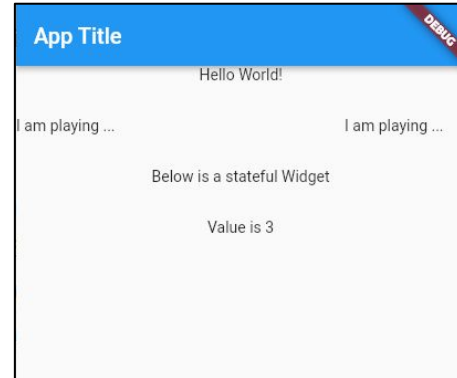
To pass an argument into the constructor of the Stateful Widget

```
import 'package:flutter/material.dart';
import
'package:reminder_flutter_hello_world_two/widgets/myStatefulWidget.d
art';
import
'package:reminder_flutter_hello_world_two/widgets/myWidget.dart';

void main() {
  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: MyStatefulWidget(myStart:3),
    );
  }
}
```



Stateless & Stateful Widgets for

- Stateful Widget

initState is invoked only
one time once the
widget is initialized.

Vs

The method **build** is
called often whenever a
refresh is required.

```
import 'package:flutter/material.dart';

class MyStatefulWidget extends StatefulWidget {
  var myStart = 0;
  MyStatefulWidget({super.key, this.myStart = 0});

  @override
  _MyStatefulWidgetState createState() => _MyStatefulWidgetState();
}

class _MyStatefulWidgetState extends State<MyStatefulWidget> {
  int increment = 0;
  @override
  void initState() {
    super.initState();
    increment = widget.myStart;
  }
  @override
  Widget build(BuildContext context) {
    return Column(children: [
      Text('Value is $increment'),
      SizedBox(height: 10),
      ElevatedButton(
        onPressed: () {
          increment = increment + 1;
          setState(() {});
        },
        child: const Text('Increment'))
    ]);
  }
}
```

Stateless & Stateful Widgets for

- Stateful Widget

`setState(() {});`

Is called to invoke the build method of the stateful widget state.

=

Refresh the UI of the widget

```
import 'package:flutter/material.dart';

class MyStatefulWidget extends StatefulWidget {
  var myStart = 0;
  MyStatefulWidget({super.key, this.myStart = 0});

  @override
  _MyStatefulWidgetState createState() => _MyStatefulWidgetState();
}

class _MyStatefulWidgetState extends State<MyStatefulWidget> {
  int increment = 0;
  @override
  void initState() {
    super.initState();
    increment = widget.myStart;
  }
  @override
  Widget build(BuildContext context) {
    return Column(children: [
      Text('Value is $increment'),
      SizedBox(height: 10),
      ElevatedButton(
        onPressed: () {
          increment = increment + 1;
          setState(() {});
        },
        child: const Text('Increment'))
    ]);
  }
}
```

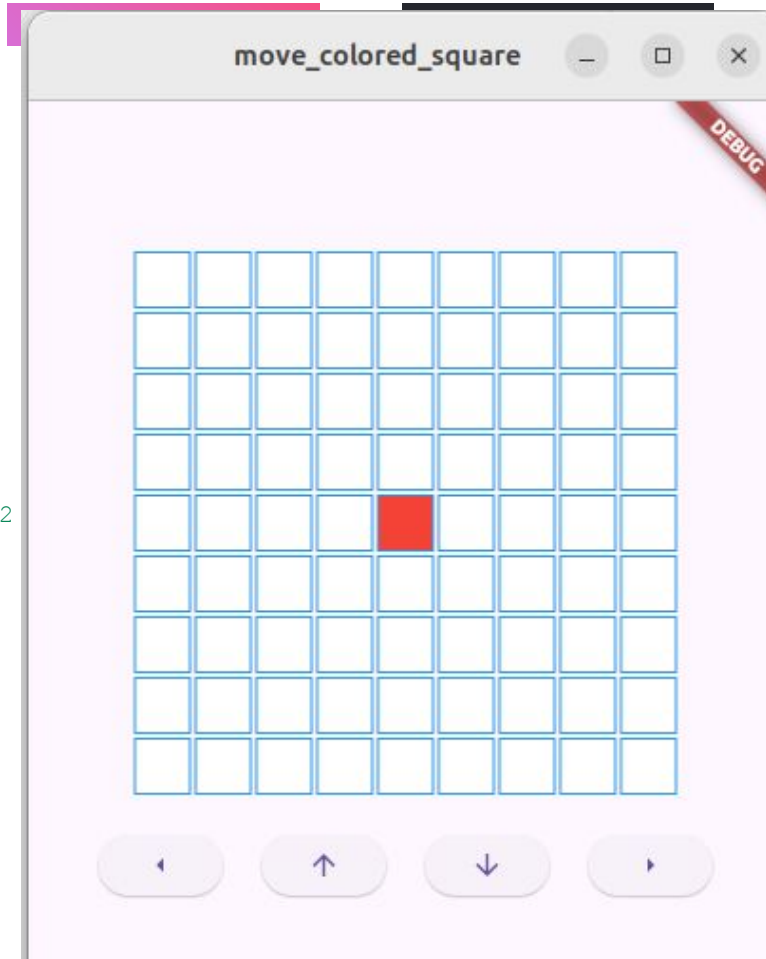


Move Colored Square

```

return Scaffold (
  body: Center (
    child: Column (
      mainAxisAlignment: MainAxisAlignment.center,
      crossAxisAlignment: CrossAxisAlignment.center,
      children: [
        for (int r = 0; r < board_size; r++)
          Row (
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              for (int c = 0; c < board_size; c++)
                Container (
                  width: board_cell_width,
                  height: board_cell_height,
                  margin: const EdgeInsets.all(1),
                  decoration: BoxDecoration (
                    color:
                      r == board_size ~/ 2 && c == board_size ~/ 2
                        ? Colors.red
                        : Colors.white,
                    border: Border.all(color: Colors.blue),
                  ),
                ),
              ),
            ],
          ),
        SizedBox (height: 20),
        Row (
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            ElevatedButton (
              onPressed: () {},
              child: const Icon(Icons.arrow_left),
            ),
            SizedBox (width: 20),
            ElevatedButton (

```

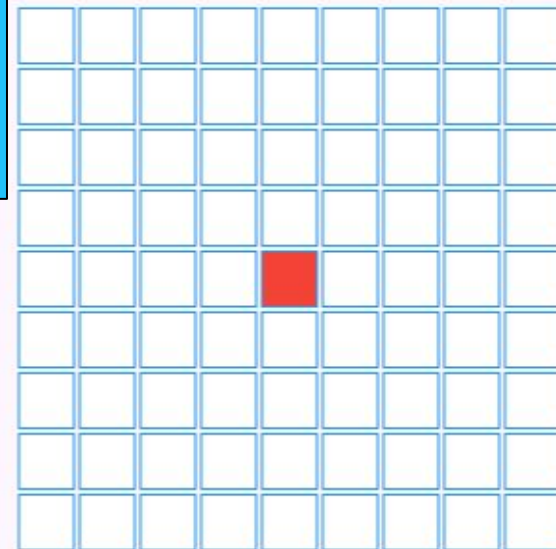


```

return Scaffold(
  body: Center(
    child: Column(
      mainAxisAlignment: MainAxisAlignment.center,
      crossAxisAlignment: CrossAxisAlignment.center,
      children: [
        for (int r = 0; r < board_size; r++)
          Row(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              for (int c = 0; c < board_size; c++)
                Container(
                  width: board_cell_width,
                  height: board_cell_height,
                  margin: const EdgeInsets.all(5),
                  decoration: BoxDecoration(
                    color:
                      r == board_size ~/ 2 && c == board_size ~/ 2
                        ? Colors.red
                        : Colors.white,
                    border: Border.all(color: Colors.blue),
                  ),
                ),
            ],
          ),
        SizedBox(height: 20),
        Row(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            ElevatedButton(
              onPressed: () {},
              child: const Icon(Icons.arrow_left),
            ),
            SizedBox(width: 20),
            ElevatedButton(

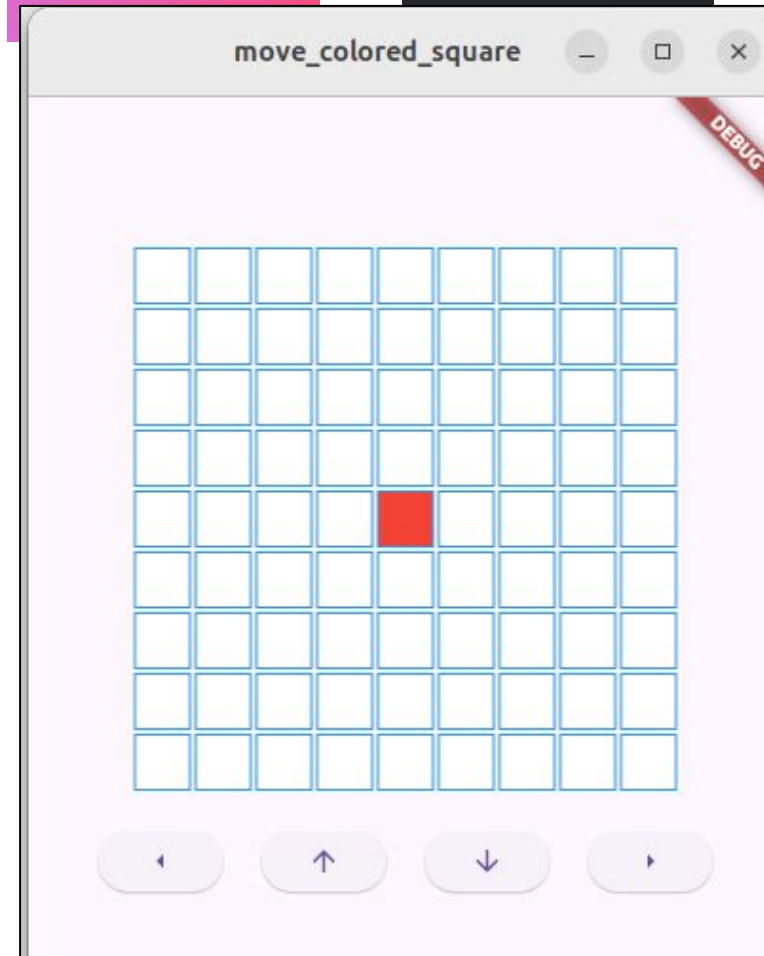
```

How to move the
red Square ?



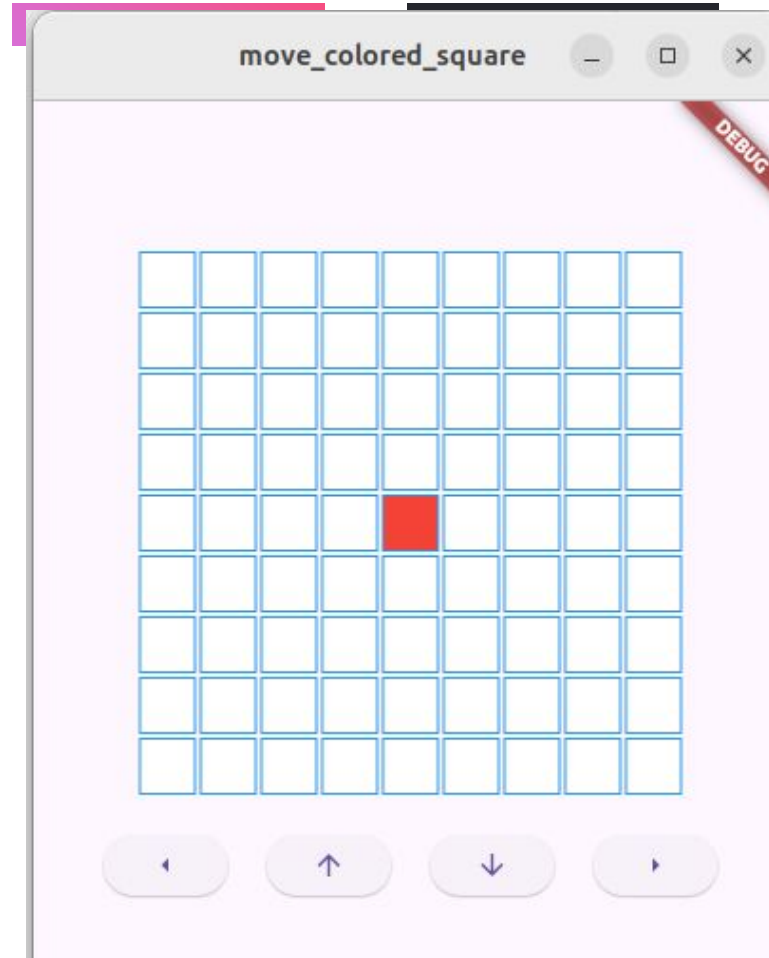
Stateful : Move Colored Square

- Create two variables:
 - current_row
 - current_column
- On events:
 - Up : decrement row
 - Left : increment row




```
class _BoardSquareMoveState extends State<BoardSquareMove> {  
  var current_row = board_size ~/ 2;  
  var current_col = board_size ~/ 2;  
  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  

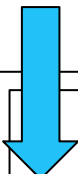
```



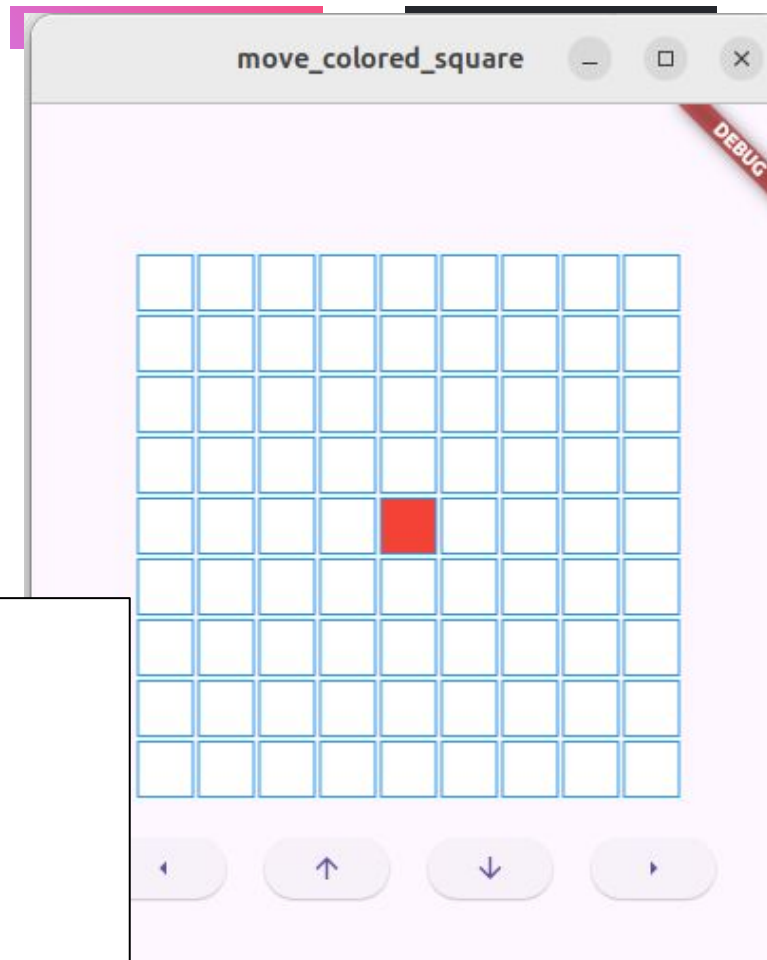
```
class _BoardSquareMoveState extends State<BoardSquareMove> {
  var current_row = board_size ~/ 2;
  var current_col = board_size ~/ 2;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
```

```
      ElevatedButton (
        onPressed: () {},
        child: const Icon(Icons.arrow_left),
      ),
```



```
      ElevatedButton(
        onPressed: () {
          if(current_col > 0){
            current_col--;
            setState(() {});
          }
        },
        child: const Icon(Icons.arrow_left),
```





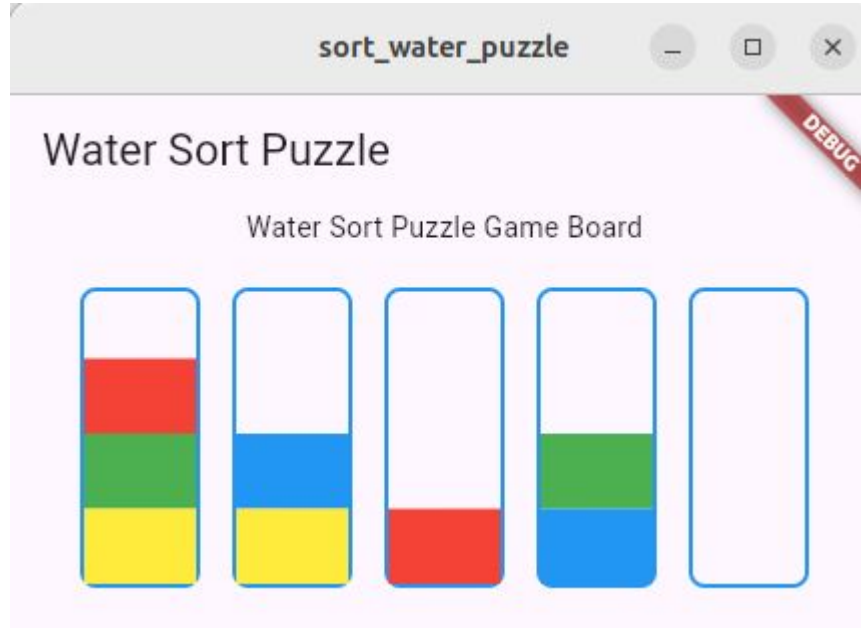
Water Sort

```

List bars = [
  [Colors.red, Colors.green],
  [Colors.blue, Colors.yellow],
  [Colors.red],
  [],
];

Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Water Sort Puzzle')),
    body: Center(
      child: Column(
        children: [
          Text('Water Sort Puzzle Game Board'),
          SizedBox(height: 20),
          Row(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              for (int b = 0; b < bars.length; b++)
                Container(
                  width: 60,
                  margin: EdgeInsets.symmetric(horizontal: 8),
                  height: 150,
                  decoration: BoxDecoration(
                    border: Border.all(color: Colors.blue, width: 2),
                    borderRadius: BorderRadius.circular(8),
                  ),
                  child: Column(
                    mainAxisAlignment: MainAxisAlignment.end,
                    children: [
                      for (int i = 0; i < bars[b].length; i++)
                        Container(width: 60, height: 150 / 4, color: bars[b][i]),
                    ],
                  ),
                ),
            ],
          ),
        ],
      ),
    ),
  );
}

```

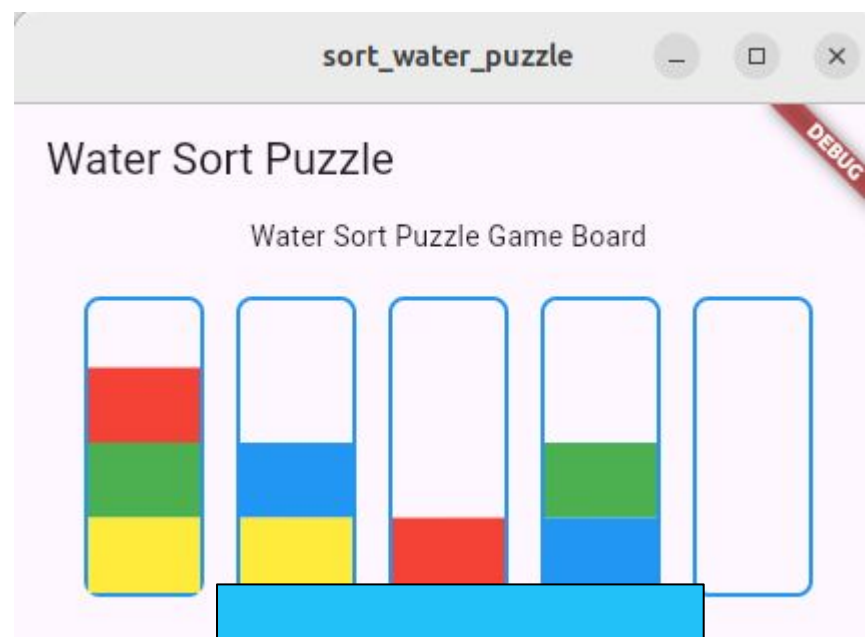


```

List bars = [
  [Colors.red, Colors.green],
  [Colors.blue, Colors.yellow],
  [Colors.red],
  [],
];

Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Water Sort Puzzle')),
    body: Center(
      child: Column(
        children: [
          Text('Water Sort Puzzle Game Board'),
          SizedBox(height: 20),
          Row(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              for (int b = 0; b < bars.length; b++)
                Container(
                  width: 60,
                  margin: EdgeInsets.symmetric(horizontal: 8),
                  height: 150,
                  decoration: BoxDecoration(
                    border: Border.all(color: Colors.blue, width: 2),
                    borderRadius: BorderRadius.circular(8),
                  ),
                  child: Column(
                    mainAxisAlignment: MainAxisAlignment.end,
                    children: [
                      for (int i = 0; i < bars[b].length; i++)
                        Container(width: 60, height: 150 / 4, color: bars[b][i],

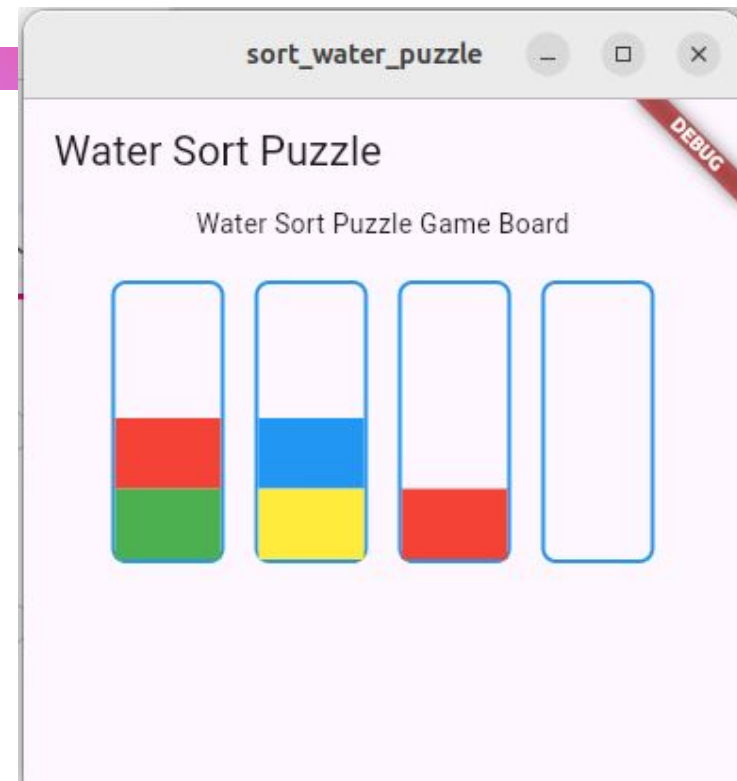
```



How to move the
**Water between
Jars ?**

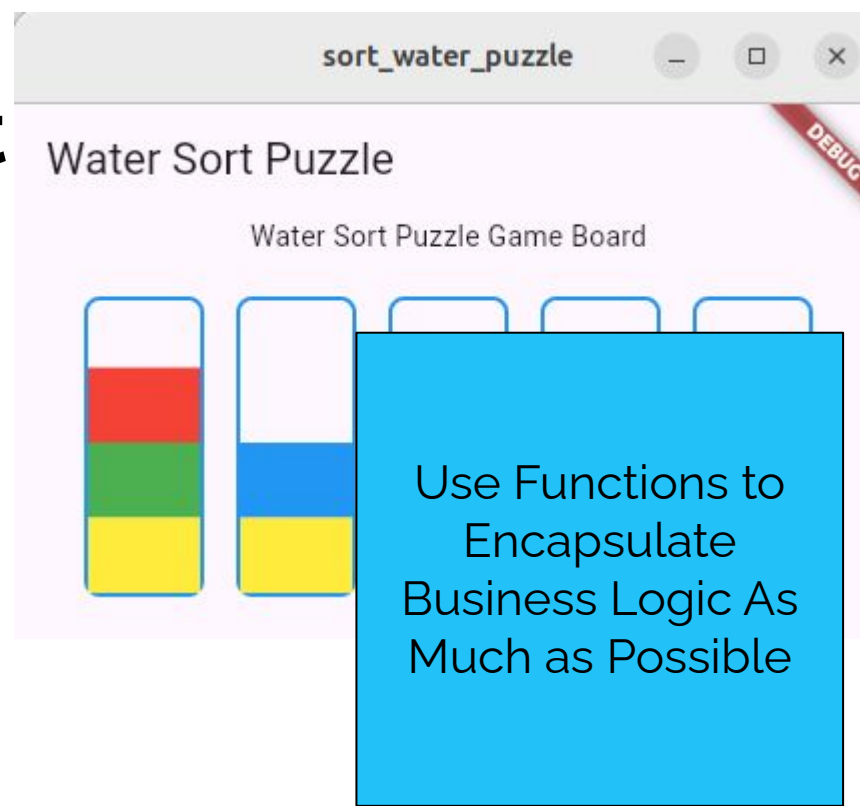
Stateful : Water Sort Puzzle

- Create two variables:
 - from_jar
 - to_jar
- On events:
 - GestureDetector : OnTap
 - if from_jar== -1 : set it to current Jar
 - Else, update to_jar, call another function to **MoveWater**



Stateful : Water Sort Puzzle

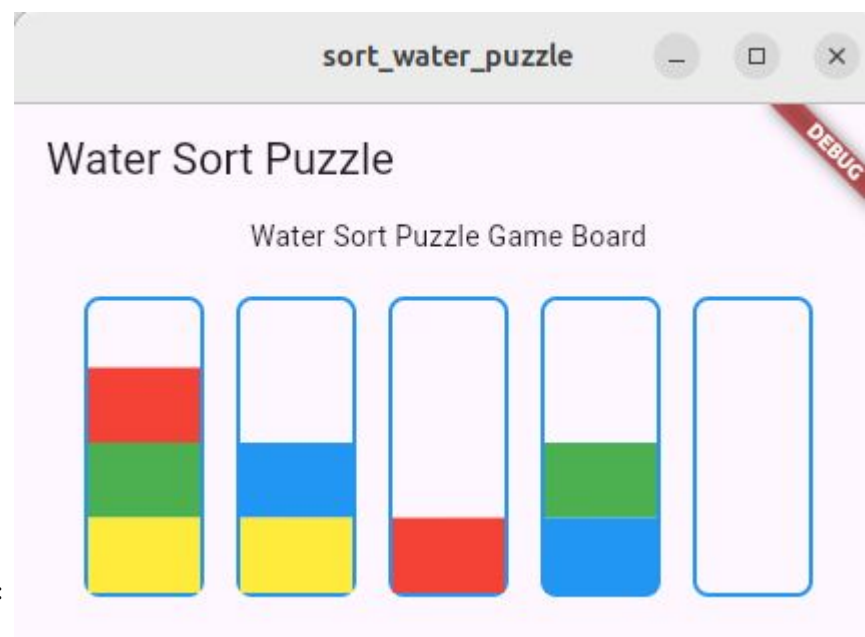
- Create two variables:
 - from_jar
 - to_jar
- On events:
 - GestureDetector : OnTap
 - if from_jar== -1 : set it to current Jar
 - Else, update to_jar, call another function to **MoveWater**



```

Text('Water Sort Puzzle Game Board'),
SizedBox(height: 20),
Row(
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    for (int b = 0; b < bars.length; b++)
      GestureDetector(
        onTap: () {
          handleClickJar(b);
        },
        child: Container(
          width: 60,
          margin: EdgeInsets.symmetric(horizontal:
            height: 150,
            decoration: BoxDecoration(
              border: Border.all(color: Colors.blue, width: 2),
              borderRadius: BorderRadius.circular(8),
            ),
            child: Column(
              mainAxisAlignment: MainAxisAlignment.end,
              children: [

```

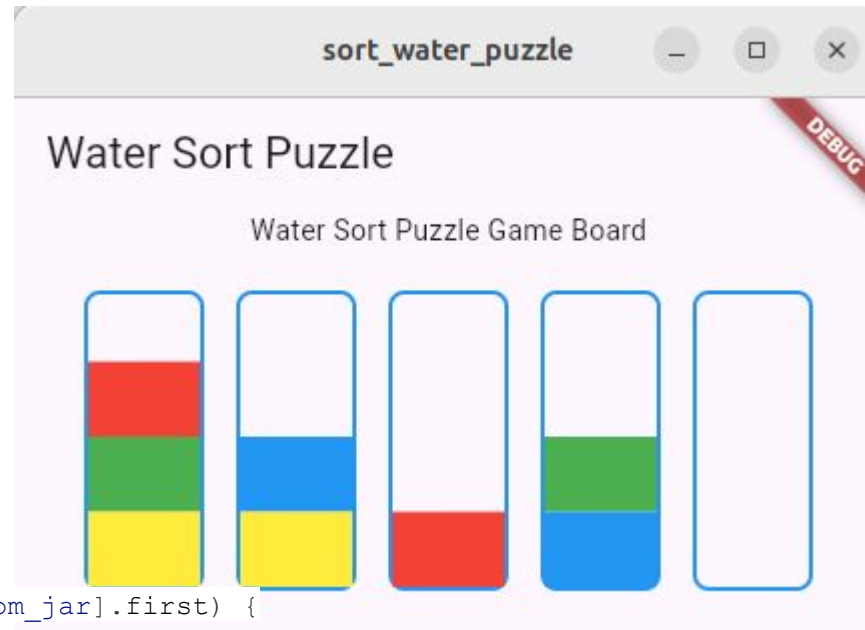


**Avoid mixing
Business Logic code
with UI Building Code**


```

var from_jar = -1;
var to_jar = -1;
bool handleClickJar(int jarIndex) {
    if (from_jar == -1) {
        from_jar = jarIndex;
    } else {
        to_jar = jarIndex;
        moveWaterIfPossible();
    }
    print('from: $from_jar, to: $to_jar');
    return true;
}
bool moveWaterIfPossible() {
    if (bars[from_jar].isEmpty) {
        resetFromTo();
        return false;
    }
    if (bars[to_jar].isEmpty || bars[to_jar].first == bars[from_jar].first) {
        if (bars[to_jar].length < 4) {
            bars[to_jar].insert(0, bars[from_jar][0]);
            bars[from_jar].removeAt(0);
            setState(() {});
        }
    }
    resetFromTo();
    return true;
}
bool resetFromTo() {
    from_jar = -1;
    to_jar = -1;
    return true; }

```



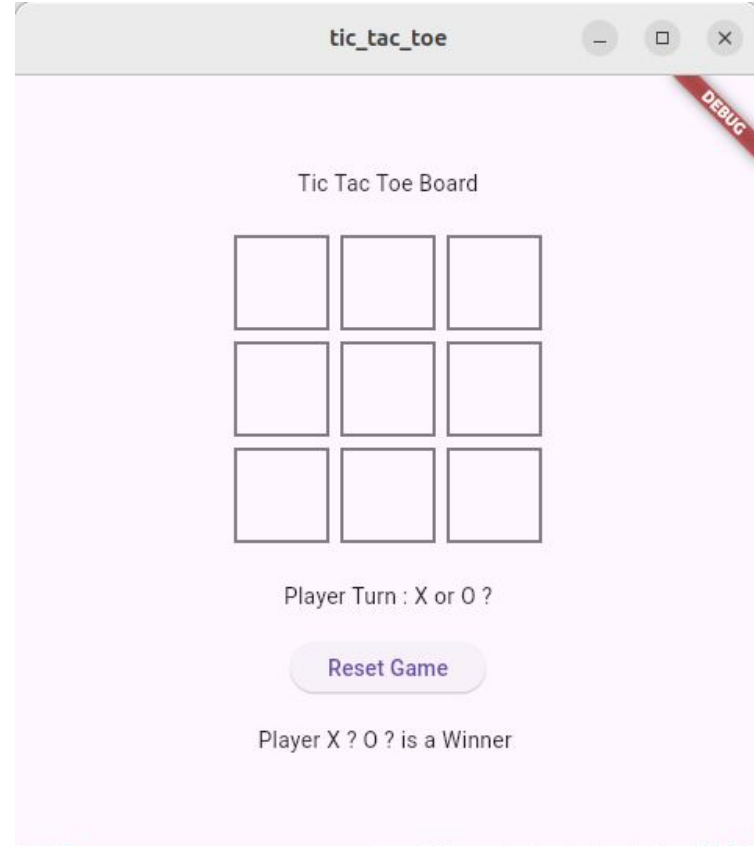


Tic Tac Toe

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    body: Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          Text('Tic Tac Toe Board'),
          SizedBox(height: 20),
          for (int i = 0; i < 3; i++)
            Row(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                for (int j = 0; j < 3; j++)
                  Container(
                    width: 60,
                    height: 60,
                    margin: EdgeInsets.all(4),
                    decoration: BoxDecoration(
                      border: Border.all(color: Colors.black),
                    ),
                    child: Center(child: Text('')),
                  ),
                ],
            ),
          SizedBox(height: 20),
          Text('Player Turn : X or O ?'),
          SizedBox(height: 20),
          ElevatedButton(onPressed: () {}, child: Text('Reset Game')),
          SizedBox(height: 20),
          Text('Player X ? O ? is a Winner '),
        ],
      ),
    ),
  );
}

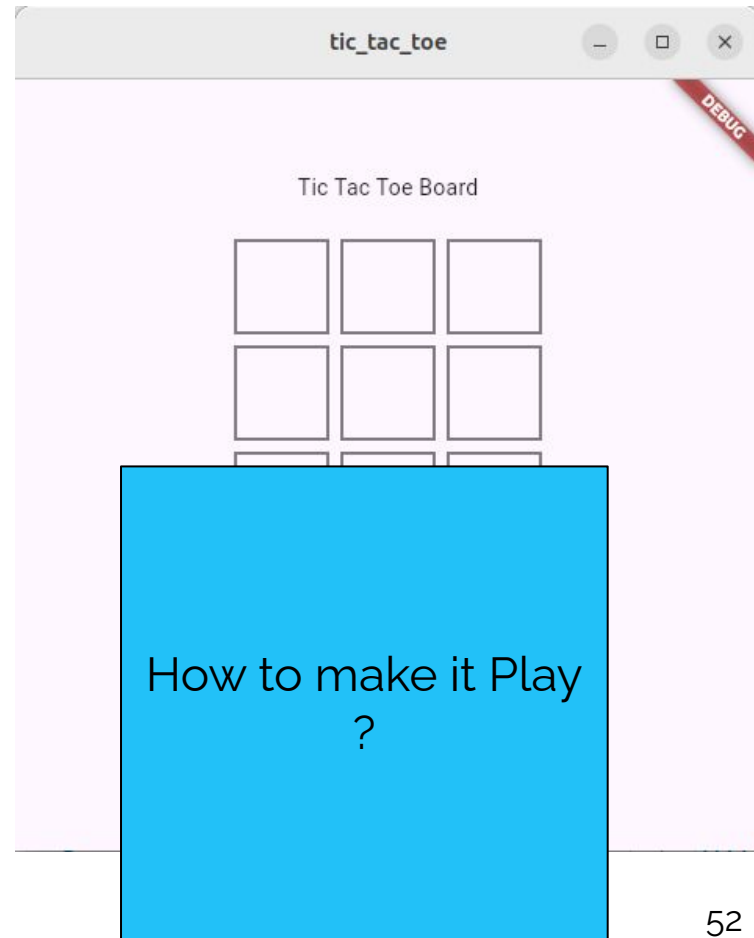
```



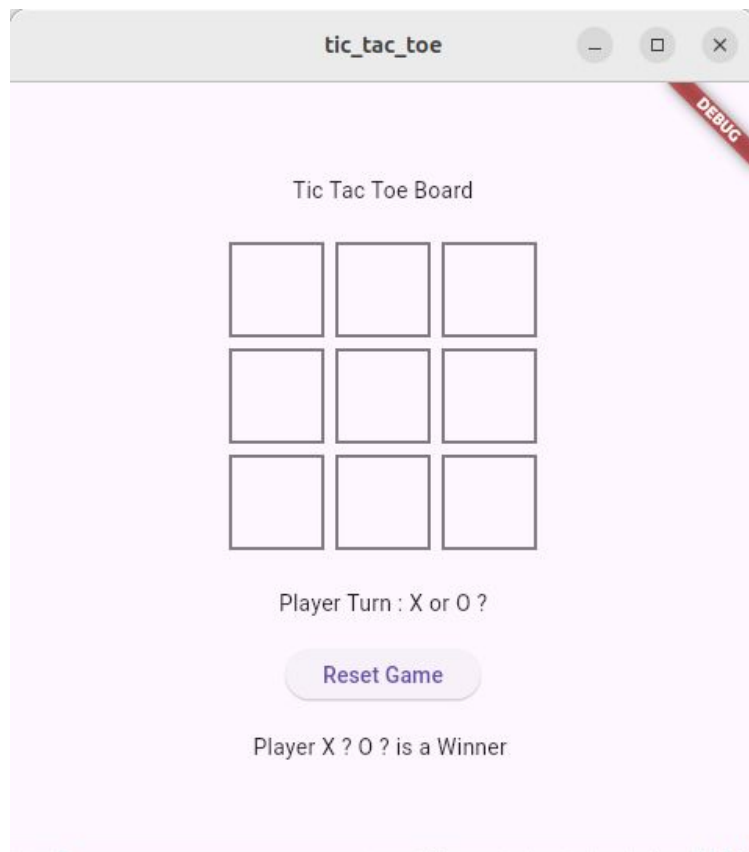
```

@override
Widget build(BuildContext context) {
  return Scaffold(
    body: Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          Text('Tic Tac Toe Board'),
          SizedBox(height: 20),
          for (int i = 0; i < 3; i++)
            Row(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                for (int j = 0; j < 3; j++)
                  Container(
                    width: 60,
                    height: 60,
                    margin: EdgeInsets.all(4),
                    decoration: BoxDecoration(
                      border: Border.all(color: Colors.black),
                    ),
                    child: Center(child: Text('')),
                  ),
                ],
            ),
          SizedBox(height: 20),
          Text('Player Turn : X or O ?'),
          SizedBox(height: 20),
          ElevatedButton(onPressed: () {}, child: Text('Reset Game')),
          SizedBox(height: 20),
          Text('Player X ? O ? is a Winner '),
        ],
      ),
    ),
  );
}

```



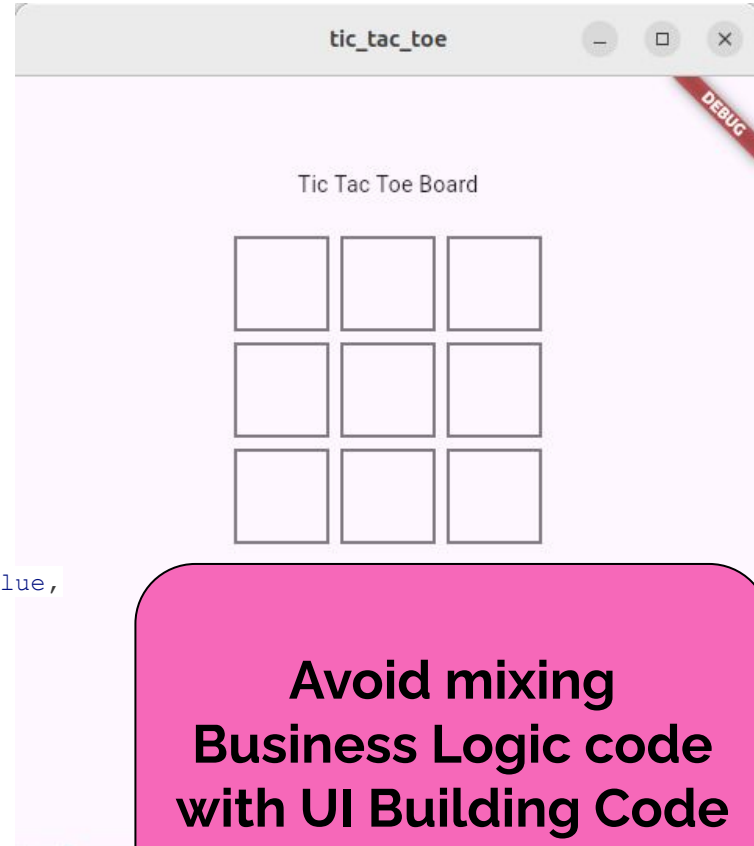
- Create variables:
 - **current_turn : String**
 - **board_data : List<List<String>>**
 - **is_game_over : bool**
- Initialize Function:
 - Board Cells $\Rightarrow$ Empty String
 - Random current_turn to either X, O
 - is_game_over : False
- Play : On Tap:
 - If Board_data Cell is empty:
 - Set its value to current_turn
 - Switch Current turn



```

for (int i = 0; i < 3; i++)
  Row(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
      for (int j = 0; j < 3; j++)
        GestureDetector (
          onTap: () {
            makePlayOnClick(i, j);
          },
          child: Container (
            width: 60,
            height: 60,
            margin: EdgeInsets.all(4),
            decoration: BoxDecoration (
              border: Border.all(color: Colors.black),
            ),
            child: Center (
              child: Text (
                board[i][j],
                style: TextStyle(
                  fontSize: 24,
                  color:
                    board[i][j] == 'X' ? Colors.red : Colors.blue,
                ),
              ),
            ),
          ),
        ),
      SizedBox(height: 20),
      Visibility(visible: !is_game_over,
        child: Text('Player Turn : $player_turn ?'),),
      Visibility(
        visible: is_game_over,
        child: ElevatedButton (
          onPressed: () {
            init_game();
          },

```



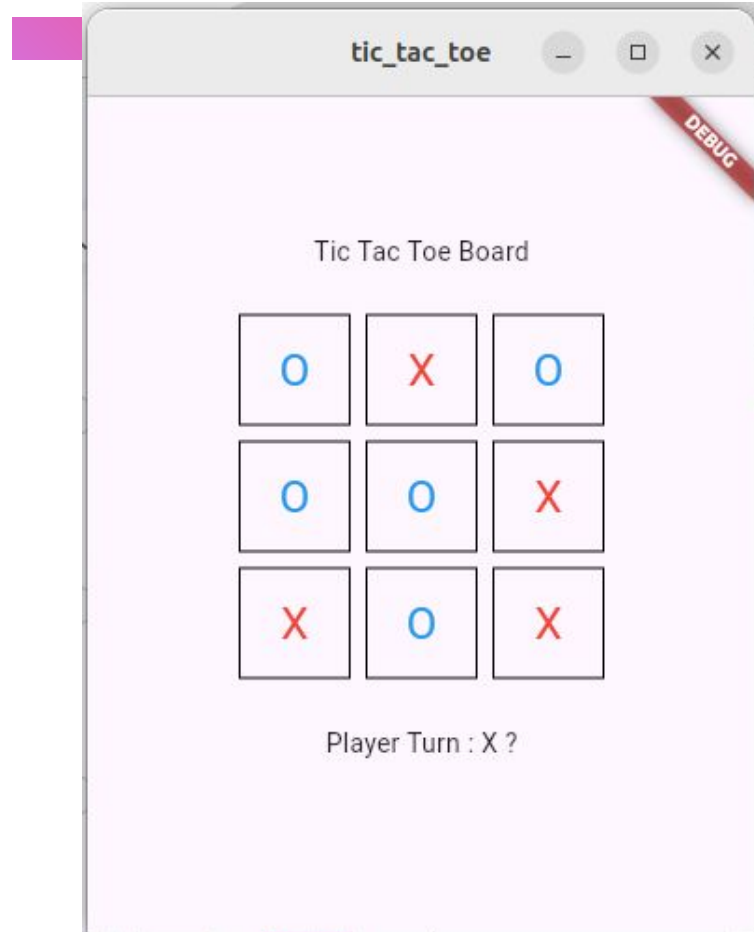
```

class TicTacToeBoardState extends State<TicTacToeBoard> {
    var is_game_over = false;
    var player_turn = Random().nextInt(2) == 0 ? 'X' : 'O';
    var board = [
        [' ', ' ', ' '],
        [' ', ' ', ' '],
        [' ', ' ', ' '],
    ];

    bool init_game() {
        is game over = false;
        player_turn = Random().nextInt(2) == 0 ? 'X' : 'O';
        board = [
            [' ', ' ', ' '],
            [' ', ' ', ' '],
            [' ', ' ', ' '],
        ];
        return true;
    }

    bool makePlayOnClick(int row, int col) {
        if (is_game_over || board[row][col] != ' ') {
            return false;
        }
        board[row][col] = player_turn;
        player turn = player_turn == 'X' ? 'O' : 'X';
        setState(() {});
        return true;
    }
}

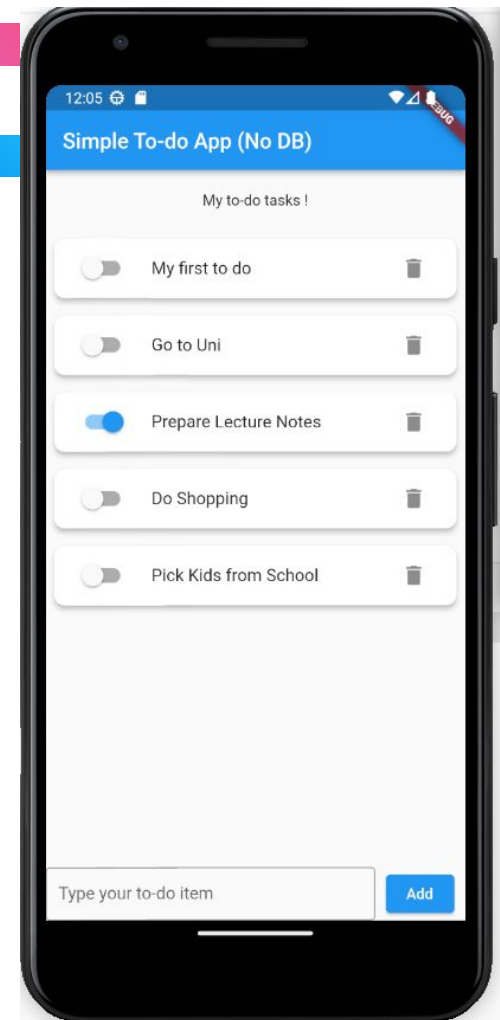
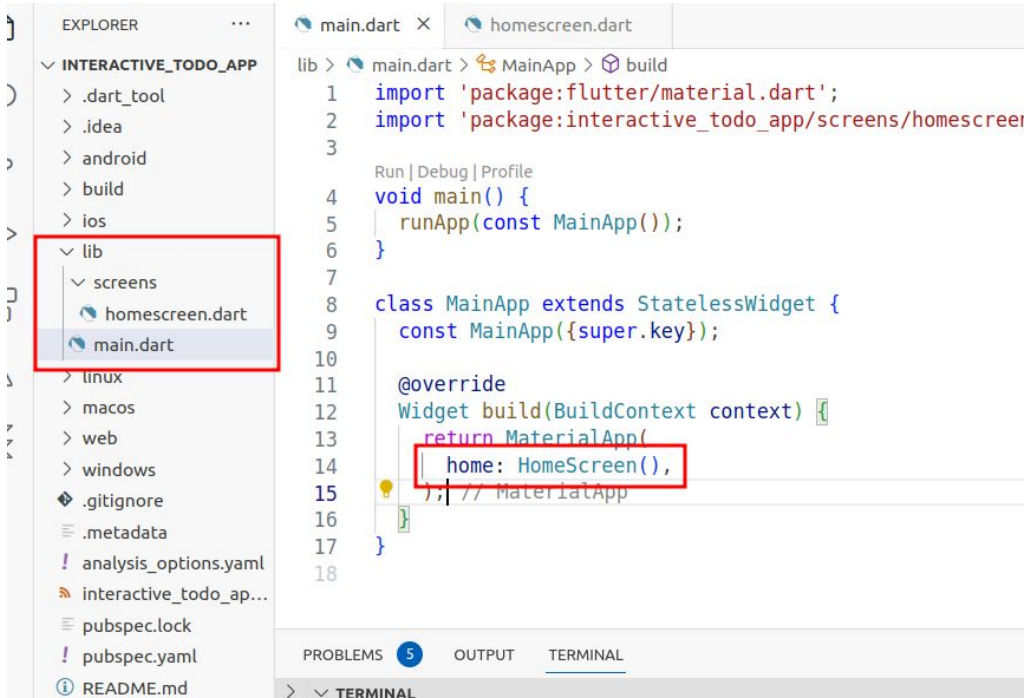
```





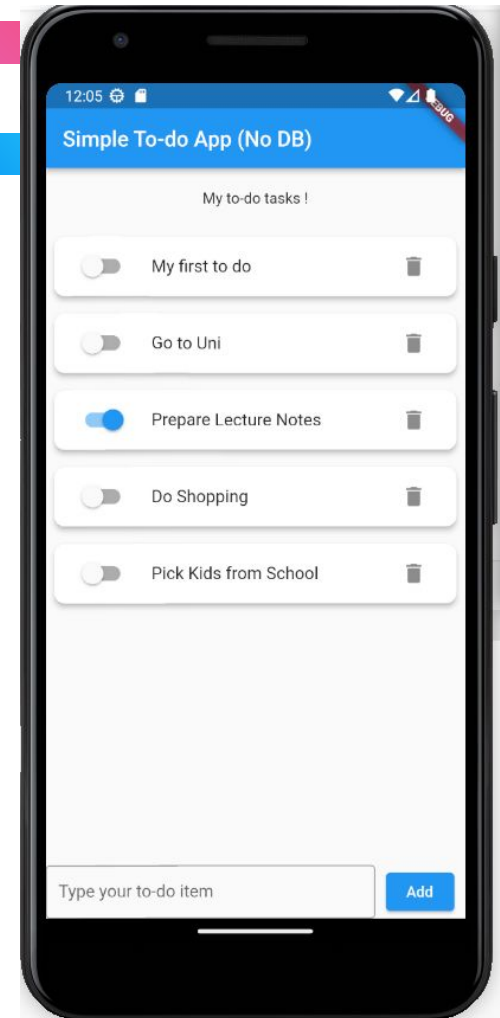
To do app

To-Do App (No DB)



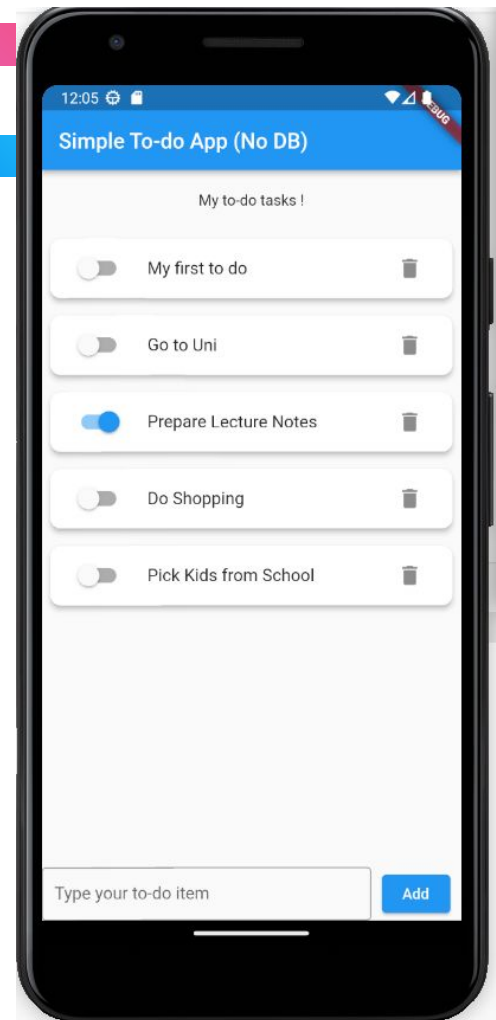
To-Do App (No DB)

```
2
3 class HomeScreen extends StatefulWidget {
4   @override
5   HomeScreenState createState() => _HomeScreenState();
6 }
7
8 class _HomeScreenState extends State<HomeScreen> {
9   List<Map> data = [
10     {'title': 'My first to do', 'done': false}
11   ];
12   String _tx title value = '';
13   final _tx title controller = TextEditingController();
14   @override
15   Widget build(BuildContext context) {
16     return Scaffold(
```



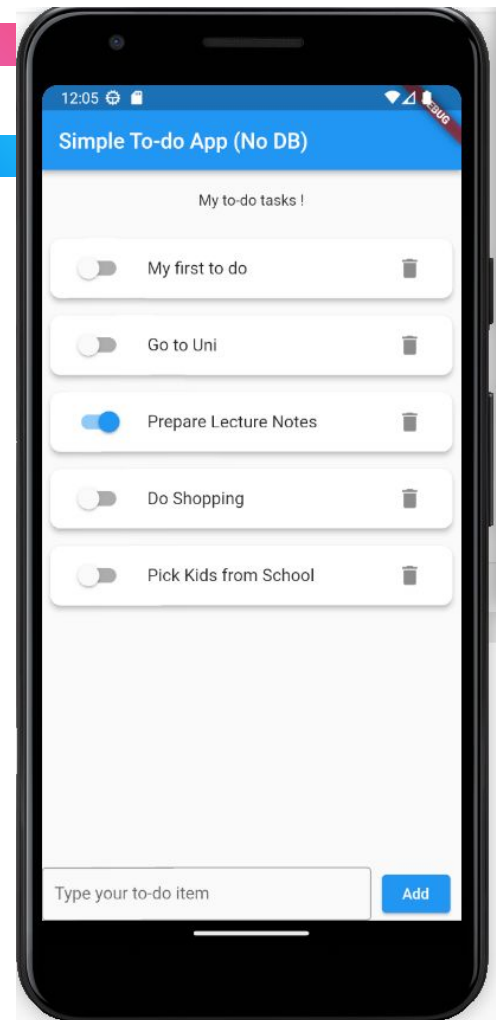
To-Do App (No DB)

```
74 | ), // Expanded
75 | const SizedBox(width: 10),
76 | ElevatedButton(
77 |   onPressed: () {
78 |     _tx_title_controller.text = '';
79 |     data.add({
80 |       'title': _tx_title_value,
81 |       'done': false,
82 |     });
83 |     setState(() {});
84 |   },
85 |   child: const Text(
86 |     "Add",
87 |   ), // Text
88 | ), // ElevatedButton
89 | const SizedBox(width: 10),
90 | ], // <Widget>[] // Row
```



To-Do App (No DB)

```
27 itemBuilder: (context, index) {
28   return Card(
29     elevation: 4,
30     margin: const EdgeInsets.all(8),
31     shape: RoundedRectangleBorder(
32       borderRadius: BorderRadius.circular(10.0),
33     ), // RoundedRectangleBorder
34     child: ListTile(
35       title: Text(data[index]['title']),
36       leading: (Switch(
37         value: data[index]['done'],
38         onChanged: (value) {
39           data[index]['done'] = value;
40           setState(() {});
41         },
42       )), // Switch
43       trailing: IconButton(
44         icon: Icon(
45           Icons.delete,
46         ), // Icon
47         onPressed: () {
48           data.removeAt(index);
49           setState(() {});
50         },
51       ), // IconButton
52     ), // ListTile
53   ), // Card
```

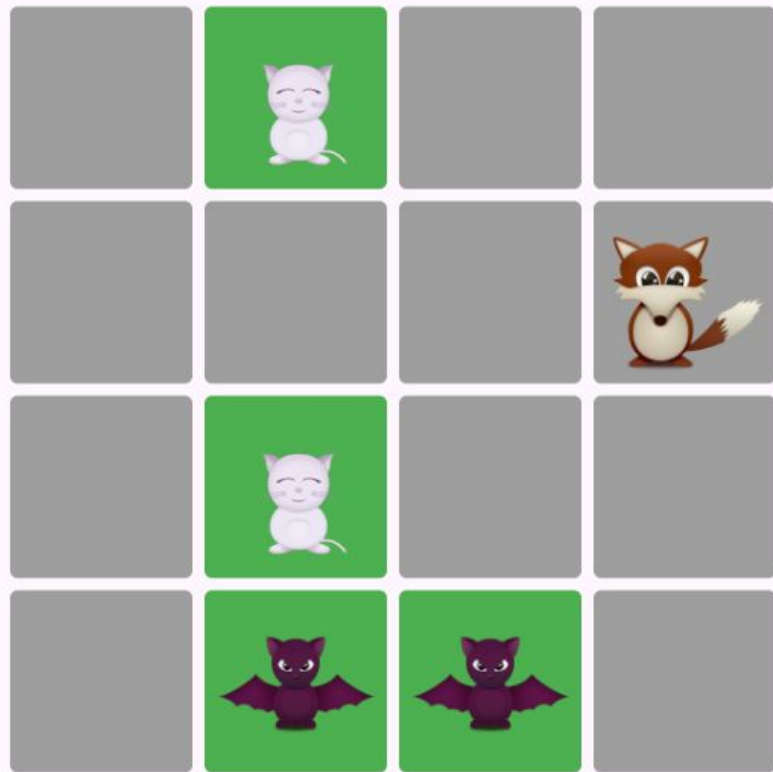




other board games

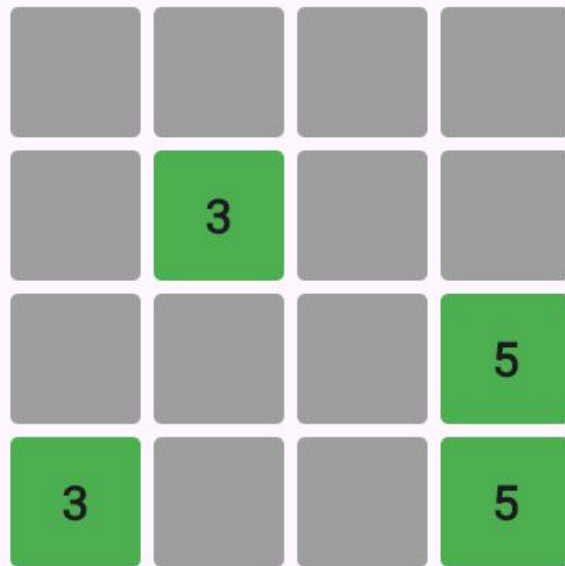
Memory Game

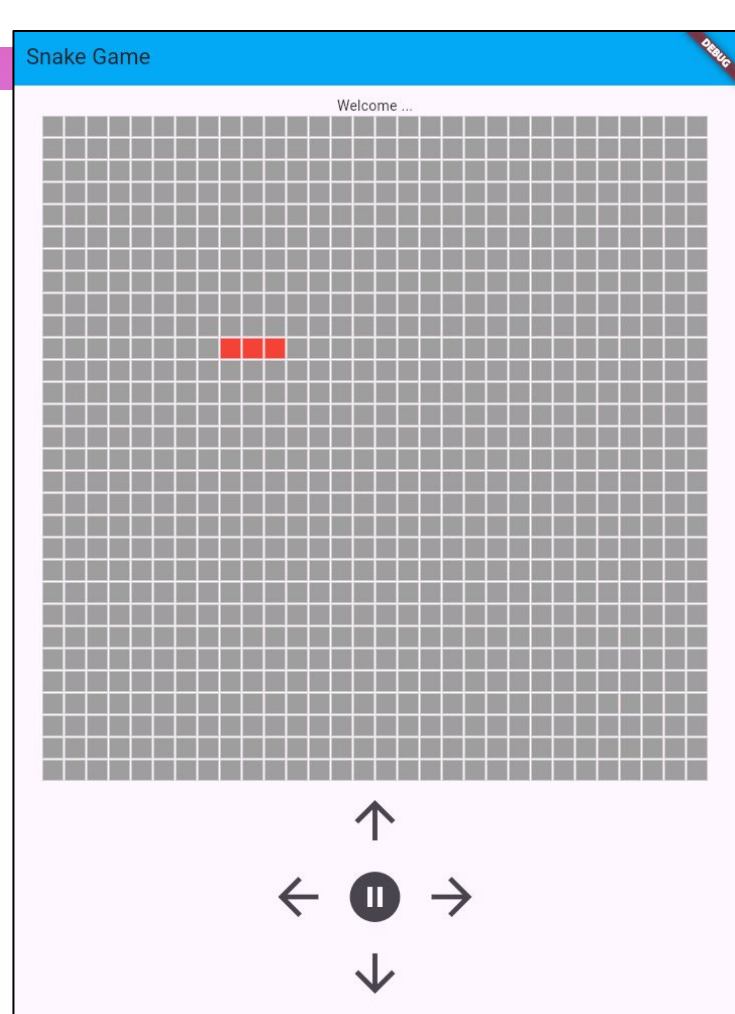
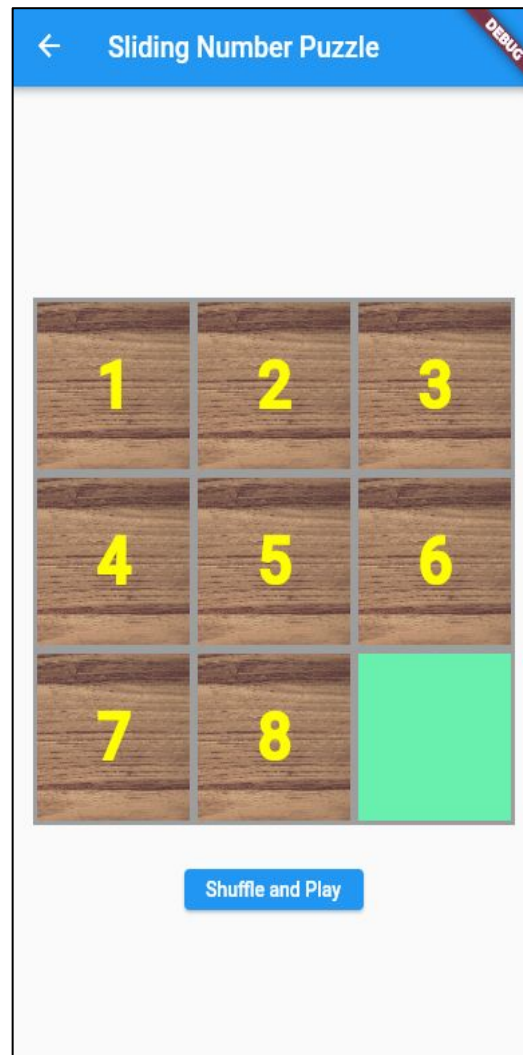
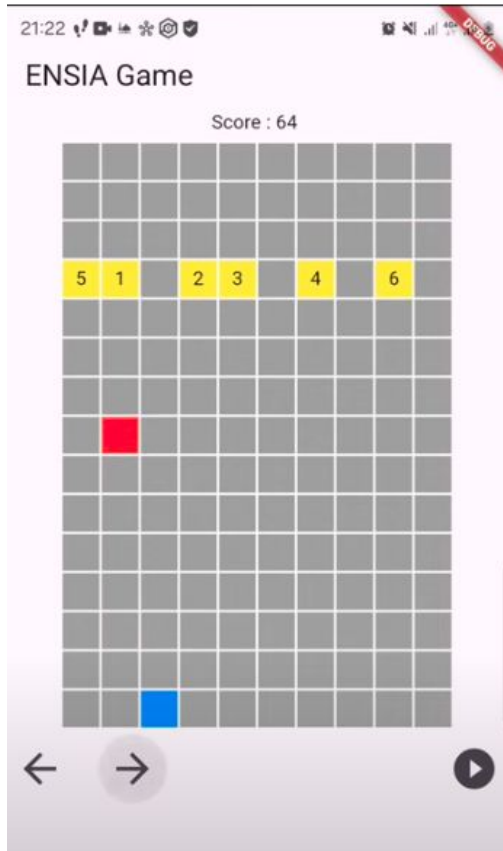
DEBUG



Memory Game

DEBUG





Section 3

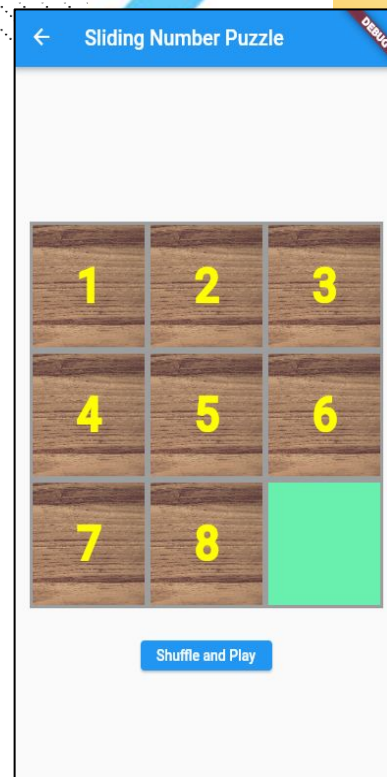
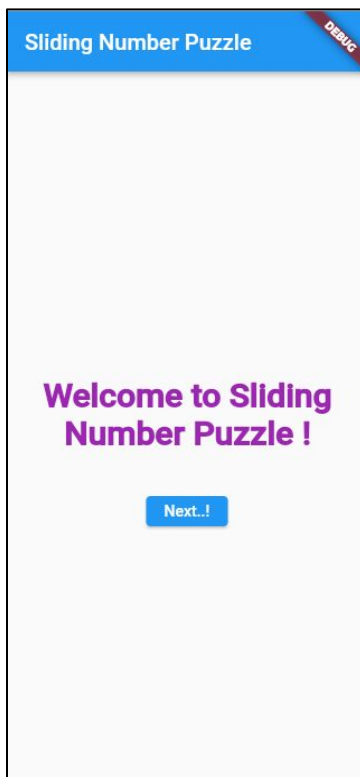
Navigation between screens

- *Go to new Screen*
- *Passing data from Parent → Child*
- *Passing data back from child → Parent*



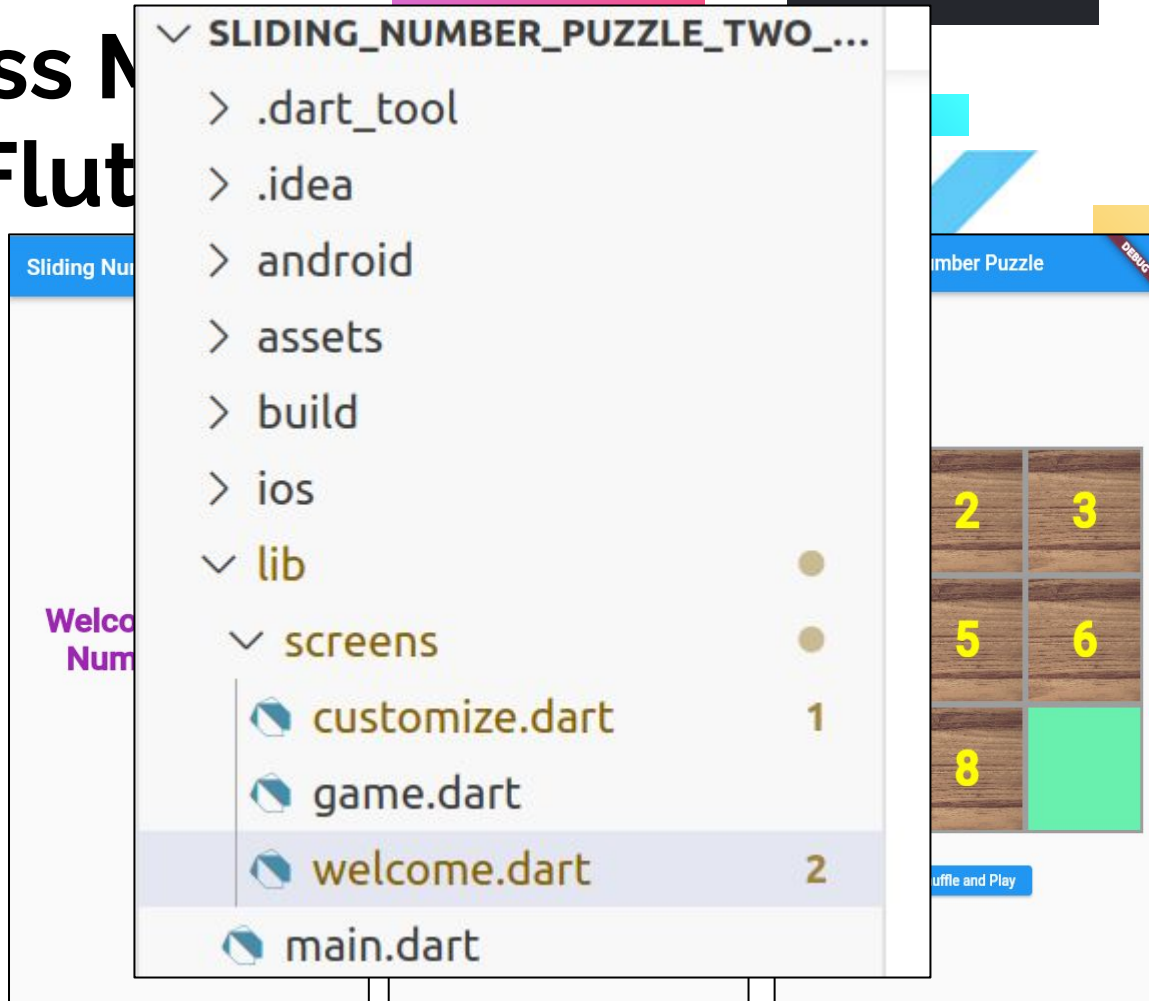
Navigation across Multiple Screens in Flutter

- **Sliding Number Puzzle :**
 - **Three Screens**
 - Welcome Screen
 - welcome.dart
 - Customize Screen
 - customize.dart
 - Game Screen
 - game.dart



Navigation across Multiple Screens in Flutter

- **Sliding Number Puzzle :**
 - **Three Screens**
 - Welcome Screen
 - welcome.dart
 - Customize Screen
 - customize.dart
 - Game Screen
 - game.dart



Navigation across Multiple Screens in Flutter



- Navigation in Flutter manages moving between screens (widgets).
- It uses a **stack** structure — new screens are pushed and popped.
- **Example** : Welcome Screen → Customize Screen → Game Screen
- Two Ways of Doing Navigation:
 1. Named Routes : Centralized and cleaner structuring
 - In static way using Routes.
 - Dynamic way using : **onGenerateRoute**
 2. Basic Navigation using **push** : simple but not scalable for large apps.



Navigating between screens

Navigation across Multiple Screens in Flutter

- **Navigation using Named Routes (static Routes Properties) :**

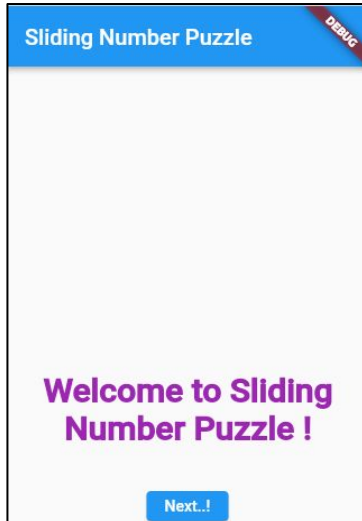
1. All Routes are defined/specified within the **routes** properties of the MaterialApp root widget
 - Define : address as a string $\Rightarrow$ Widget screen to be created upon visiting this address.
2. For better organization of the code, the address of each screen is defined as a **static string** variable with the screen widget class
3. At any screen of the app, to navigate to a different address just call :
 - **Navigator.pushNamed(context, 'address_given');**

Navigating to the Main Screen

1

- Sliding Number Puzzle :

- main.dart File :



```
import 'package:flutter/material.dart';
import
'package:sliding_number_puzzle_two_screens/screens/customize.dart';
import 'package:sliding_number_puzzle_two_screens/screens/game.dart';
import
'package:sliding_number_puzzle_two_screens/screens/welcome.dart';

void main() {
  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: WelcomeScreen(),
      routes: {
        HomeScreen.pageRoute: (ctx) => HomeScreen(),
        WelcomeScreen.pageRoute: (ctx) => WelcomeScreen(),
        CustomizeScreen.pageRoute: (ctx) => CustomizeScreen(),
      },
    );
  }
}
```

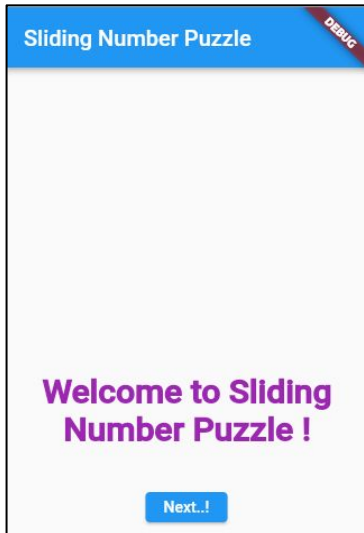
**First Step : Define
Routes within the
MaterialApp
Widget**

Navigation

2

- Sliding Number Puzzle :

- Welcome Screen :



```
import 'package:flutter/material.dart';
import 'package:sliding_number_puzzle_two_screens/screens/customize.dart';

class WelcomeScreen extends StatelessWidget {
  static const String pageRoute = '/welcomescreen';

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Sliding Number Puzzle')),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          crossAxisAlignment: CrossAxisAlignment.center,
          children: [
            const Text(
              'Welcome to Sliding Number Puzzle !',
              textAlign: TextAlign.center,
              style: TextStyle(
                color: Colors.purple,
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

Address of each screen is defined as a **static string** variable with the screen widget class

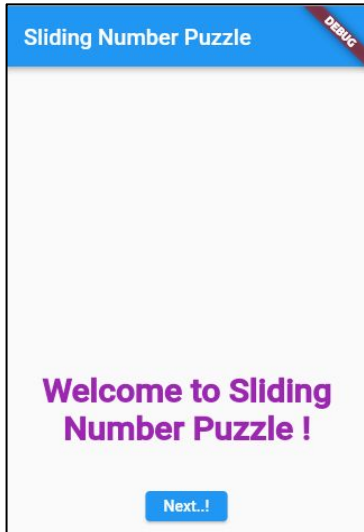
/strong convention : start with /

Navigation

3

- Sliding Number Puzzle

- Welcome Screen



```
import 'package:flutter/material.dart';
import 'package:sliding_number_puzzle_two_screens/screens/customize_screen.dart';

class WelcomeScreen extends StatelessWidget {
  static const String pageRoute = '/welcomescreen';

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Sliding Number Puzzle')),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          crossAxisAlignment: CrossAxisAlignment.center,
          children: [
            const Text(
              'Welcome to Sliding Number Puzzle !',
              textAlign: TextAlign.center,
              style: TextStyle(
                color: Colors.purple,
                fontWeight: FontWeight.bold,
                fontSize: 30),
            ),
            const SizedBox(height: 40),
            ElevatedButton(
              onPressed: () {
                Navigator.pushNamed(context, CustomizeScreen.pageRoute);
              },
              child: Text("Next..!"),
            ),
          ],
        ),
      ),
    );
  }
}
```



Navigation across Multiple Screens in Flutter

- **Navigation using Named Routes (Dynamic way) :**

1. Using the property **onGenerateRoute: (settings) {}** of the MaterialApp root widget
 - Inside the function, do the business logic based on the **settings** variable to route to the correct screen:
 - Example: if user is navigated to welcome screen but not logged in, redirect them to the login screen instead.
2. Use the **Navigator.pushNamed(context, 'address_given');**

```

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  final bool isLoggedIn = false;
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Hello Navigation World ! ',
      initialRoute: '/',
      onGenerateRoute: (settings) {
        switch (settings.name) {
          case '/':
            return MaterialPageRoute( builder: (_) => HomeScreen(), );
          case '/login':
            return MaterialPageRoute(
              builder: (_) => LoginScreen(),
            );
          case '/profile':
            if (!isLoggedIn) {
              return MaterialPageRoute(builder: (_) => LoginScreen());
            }
            // Retrieve arguments safely
            final args = settings.arguments;
            if (args is Map && args.containsKey('username')) {
              return MaterialPageRoute(builder: (_) => ProfileScreen(username: args['username']));
            }
          default:
            return MaterialPageRoute(builder: (_) => NotFoundScreen(message: 'Page not found!'));
        }
      },
    );
  }
}

```

Navigation across Multiple Screens in Flutter

- **Navigation without Named Routes:**

1. Use the `MaterialPageRoute` with its **`builder`** attribute:

- **`Navigator.push(`** context,

- `MaterialPageRoute(`**

- `builder: (_) => SecondScreen()`**

- `), );`**

The value 3 needs to be sent to the child screen (next screen)

Remember : Flutter is declarative

Not imperative programming : (`someRef.getText()` no longer works)



```

children: [
  const Text(
    'Choose the game size : ', textAlign: TextAlign.center,
    style: TextStyle(
      color: Colors.purple,
      fontWeight: FontWeight.bold, fontSize: 30),
  ),
  const SizedBox(height: 40),
  SizedBox(
    width: 50,
    child: TextFormField(
      decoration: const InputDecoration(
        border: OutlineInputBorder(
          borderSide: BorderSide(color: Colors.teal)),
      ),
      initialValue: '3',
      keyboardType: TextInputType.number,
      onChanged: (newValue) {
        debugPrint('> Printing new value $newValue');
        _tx_size_value = newValue;
      },
    ),
  ),
  const SizedBox(height: 40),

```



```

    SizedBox(
      width: 50,
      child: TextFormField(
        decoration: const InputDecoration(
          border: OutlineInputBorder(
            borderSide: BorderSide(color: Colors.teal)),
        ),
        initialValue: '3',
        keyboardType: TextInputType.number,
        onChanged: (newValue) {
          debugPrint('> Printing new value $newValue');
          _tx_size_value = newValue;
        },
      ),
    ),
    const SizedBox(height: 40),
    ElevatedButton(
      onPressed: () {
        Navigator.of(context).push(MaterialPageRoute(
          builder: (context) => HomeScreen(myData: {
            'myBoardSize': _tx_size_value,
            'otherDataYouWish': 'hello',
          })));
      },
      child: Text("Start the Game !"),
    ),
  ],
),
);
}
}

```





Going back

```
ElevatedButton(  
  onPressed: () {  
    Navigator.pushNamed(context, WelcomeScreen.pageRoute);  
  },  
  child: Text('Back'),  
)
```

OR ?

```
ElevatedButton(  
  onPressed: () {  
    Navigator.pop(context);  
  },  
  child: Text('Back'),  
)
```

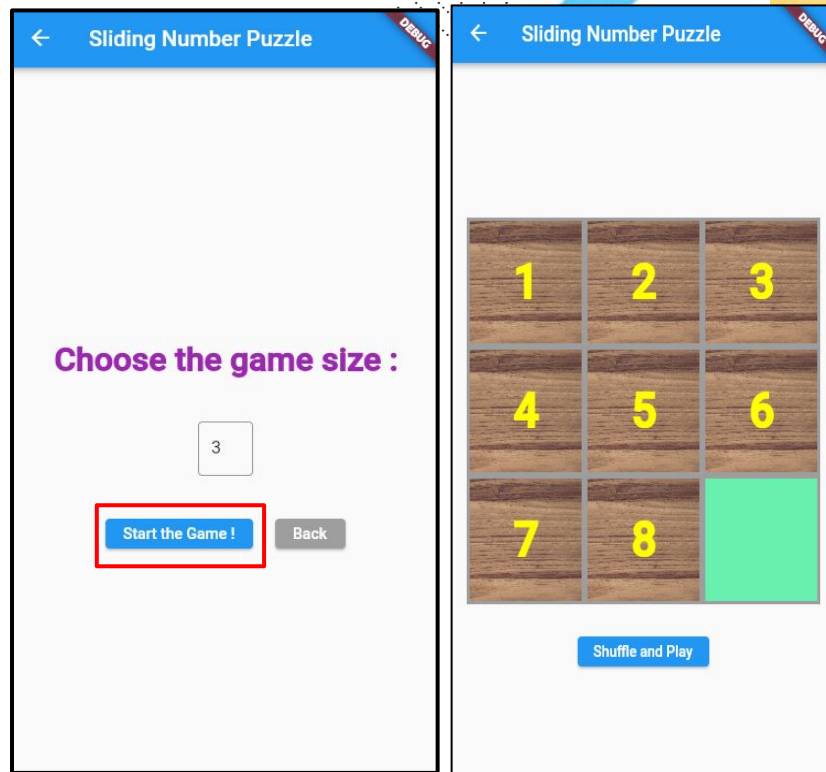




Passing data

Navigation across Multiple Screens in Flutter

- **Passing Data between Screens :**
 - The recommended way is to pass through the child constructor.
 - You may can use a Map Object to pass a variety of variables.



Navigation across Multiple Screens in Flutter

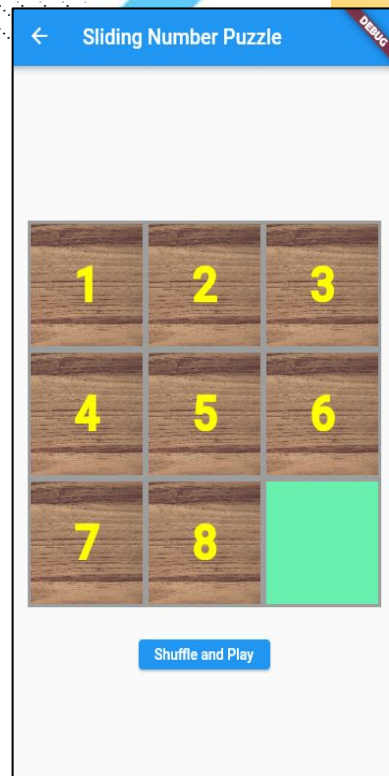
- Third Screen : game.dart

```
import 'dart:math';
import 'package:flutter/material.dart';

class HomeScreen extends StatefulWidget {
  static const String pageRoute = '/homescreen';
  final Map<String, String>? myData;

  const HomeScreen({super.key, this.myData});

  @override
  _HomeScreenState createState() => _HomeScreenState();
}
```



Navigation across Multiple Screens in Flutter

- Third Screen : game.dart

```
import 'dart:math';
import 'package:flutter/material.dart';

class HomeScreen extends StatefulWidget {
  static const String pageRoute = '/homescreen';
  final Map<String, String>? myData;
```

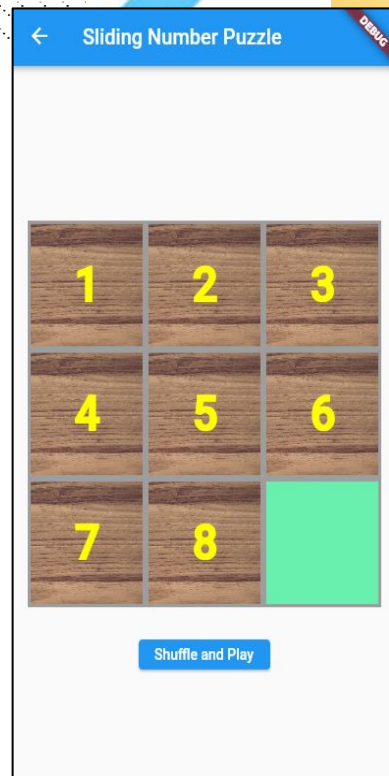
```
const
```

```
@over
```

```
_Home
```

```
}
```

How to pass the data to the Widget State Object ?



```

class _HomeScreenState extends State<HomeScreen> {
  List<List<String>> data = [];
  int gridSize = 3;
  int empty_row = 2;
  int empty_col = 2;

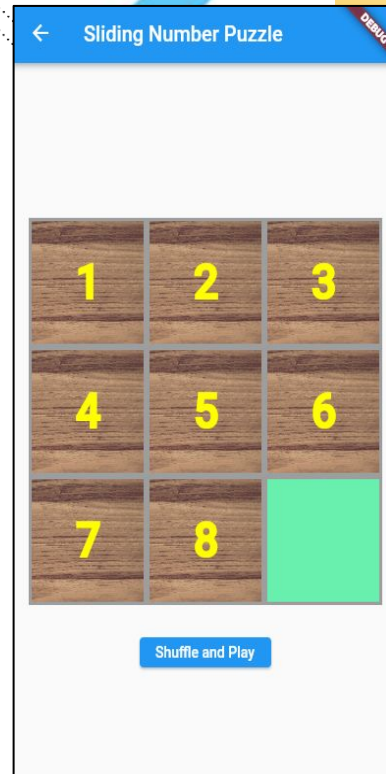
  void init_game_data() {
    try {
      gridSize = int.parse(widget.myData!['myBoardSize']);
    } catch (e) { gridSize = 3; }

    data = [];
    empty_row = gridSize - 1;
    empty_col = gridSize - 1;

    int inc = 1;
    for (int r = 0; r < gridSize; r++) {
      List<String> row = [];
      for (int c = 0; c < gridSize; c++) {
        row.add(inc.toString());
        inc = inc + 1;
      }
      data.add(row);
    }
    data[gridSize - 1][gridSize - 1] = '';
  }

  @override
  void initState() {
    super.initState();
    init_game_data();
  }
}

```



Navigation across Multiple Screens in Flutter



- **Passing Data between Screens : Other ways ?**
 - The recommended way is to pass through the child constructor.
 - You may can use a Map Object to pass a variety of variables.

Navigation across Multiple Screens in Flutter



- **Passing Data between Screens :**

- Another way

Sending

```
Navigator.pushNamed(context, someRoute, arguments: [  
    'Title of Screen',  
    ['data', 'another data'],  
    'some more'  
]),
```

Getting

```
@override  
Widget build(BuildContext context) {  
    final List args = (ModalRoute.of(context)!.settings.arguments) as List;  
    appBarTitle = args[0];  
    listOfItems = args[1];  
    infoOrState = args[2];  
}
```

Navigation across Multiple Screens in Flutter

- Passing Data Between Screens

- A

Sending

Getting

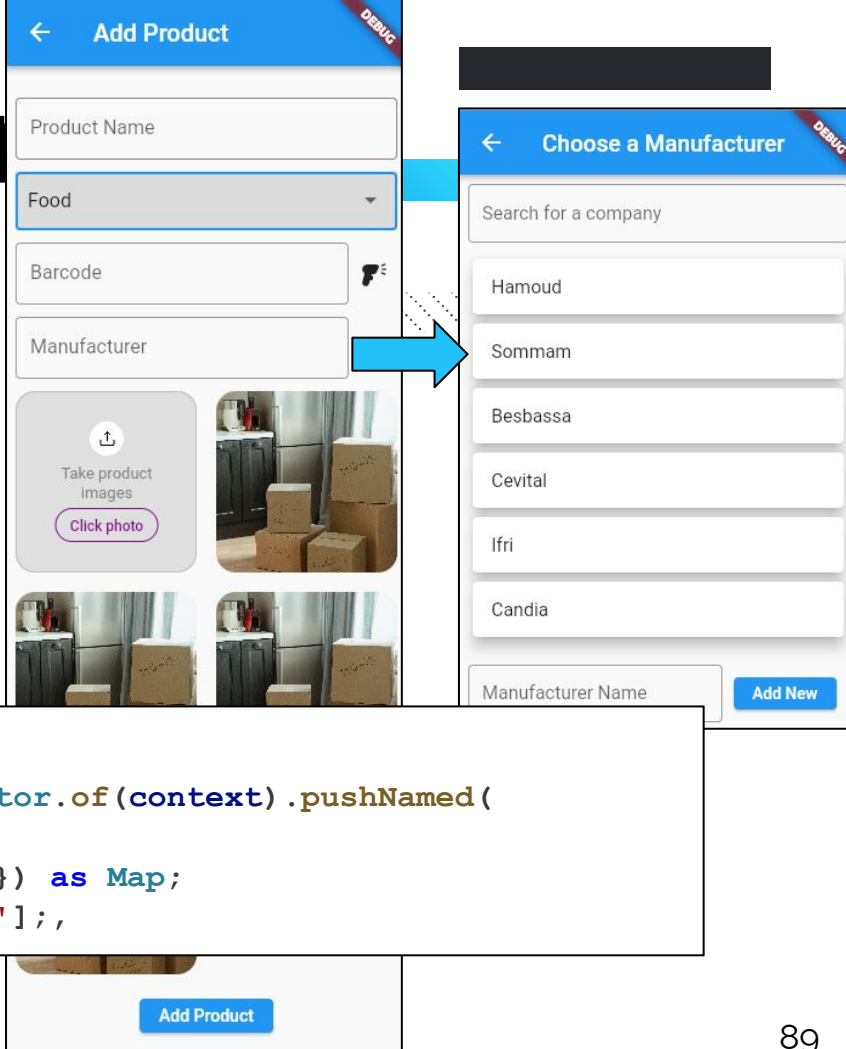
How to pass data from the child screen to the parent screen ?

```
listOfItems = args[1];  
infoOrState = args[2];
```


Navigation across Multiple Screens in Flutter

- ```
Navigator.of(context).pushNamed(
 Manufacturer.pageRoute,
 arguments: {'from_product': 1});
```

```
Map manufactData = await Navigator.of(context).pushNamed(
 Manufacturer.pageRoute,
 arguments: {'from_product': 1}) as Map;
company_name = manufactData['name'];
```

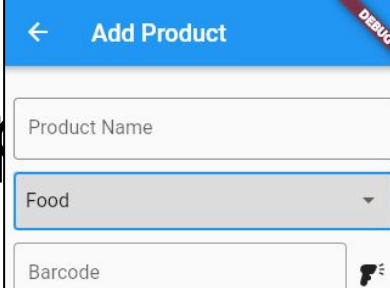


# Navigation across Multiple Screens in Flutter

- ```
onTap: () {  
    if (from_product) {  
        Navigator.pop(context, items[index]);  
    }  
},
```

○ VS

```
onTap: () async {  
    Map manufactData = await Navigator.of(context).pushNamed(  
        Manufacturer.pageRoute,  
        arguments: {'from product': 1}) as Map;  
    _tx_company_controller.text = manufactData['name'];  
    _tx_company_value = manufactData;  
},
```



Navigation across Multiple Screens in Flutter



Method	Description
<code>push()</code>	Adds a new screen
<code>pushReplacement()</code>	Replaces current
<code>pushAndRemoveUntil()</code>	Clears stack

Section 4

To read more on ..





More concepts to explore..

- Life cycle of a Widget
-

Lecture Demo Apps

- New Materials:
 - https://drive.google.com/file/d/12u8tD0g84pX5YD6WQMFOyLO5_UZWYy81S/view?usp=sharing
- Old Materials
 - Reminder Examples
 - <https://www.dropbox.com/scl/fo/ddnkks2s0k9qfwglfx8ru/h?rlkey=cg72kn3tsqjicthtgu4wekhp3&dl=0>
 - <https://www.dropbox.com/scl/fo/hm3mkdvnhibf4macm78as/h?rlkey=zga7oxols3l72a82qt5kspui1&dl=0>
 - Tic Tac Toe Game
 - Without Business Logic:
 - <https://www.dropbox.com/scl/fo/oskfiozepgyppc6z3lfmo/h?rlkey=ov2gwcgcw7w6nbstsqt58f78&dl=0>
 - With Business Logic
 - <https://www.dropbox.com/scl/fo/wv42hnx4r83g7huzhuc6e/h?rlkey=k4cx70oxnynv7cq5wclwcyndm&dl=0>
 - To Do App (No database)
 - <https://www.dropbox.com/scl/fo/o84pyfe0m98gnay5mlynz/ADynBAPxTPnueqV6Ro52hlo?rlkey=672nqxwkxi8lyppxigwvomqn7&st=i8qb9g1d&dl=0>

Resources

- <https://docs.flutter.dev/cookbook/navigation/named-routes>
- <https://docs.flutter.dev/ui/navigation>
- <https://docs.flutter.dev/cookbook/design/tabs>